Federico Costa e Emanuele De Simone



POLITECNICO
MILANO 1863

Requirement Analysis and Specification Document

Deliverable:	RASD
Title:	Requirement Analysis and Verification Document
Authors:	Federico Costa e Emanuele De Simone
Version:	1.0
Date:	October-2025
Download page:	https://github.com/EmanueleDeSimone04/CostaDeSimone
Copyright:	Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Contents

Table of Contents **3**

List of Figures **5**

List of Tables **5**

1 Introduction **6**
1.1 Purpose 6
1.1.1 Goals 7
1.2 Scope 7
1.2.1 World Phenomena 7
1.2.2 Shared Phenomena 7
1.3 Definition, Acronysm, Abbreviations - Glossary 9
1.4 Revision History 9
1.5 Reference Documents 9
1.6 Document Structure 9
1.6.1 Introduction 9
1.6.2 Overall Description 9
1.6.3 Specific Requirements 10
1.6.4 Formal Analysis Using Alloy 10
1.6.5 Effort Spent 10
1.6.6 References 10

2 Overall Description **11**
2.1 Product Perspective 11
2.1.1 Scenarios 11
2.1.2 Domain Class Diagram 13
2.1.3 State Diagrams 14
2.2 Product Functions 14
2.3 User Characteristics 17
2.4 Assumptions, Dependencies and Constraints 17

3 Specific Requirements **18**
3.1 External Interface Requirements 18
3.1.1 User Interfaces 18
3.1.2 Hardware Interfaces 18
3.1.3 Software Interfaces 18
3.1.4 Communication Interfaces 18
3.2 Functional Requirements 19
3.2.1 Functional Requirements List 19
3.2.2 Use Cases Diagrams 20
3.2.3 Use Cases 22
3.2.4 Traceability Matrix 43
3.2.5 Goals, Requirements and Domain Assumptions Mapping 44
3.3 Performance Requirements 45
3.3.1 Latency Requirements 45
3.3.2 Storage Requirements 45
3.3.3 Scalability Requirements 45
3.3.4 Throughput Requirements 45

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.4 Design Constraints . . . 45
3.4.1 Standards compliance . . . 45
3.4.2 Hardware limitations . . . 45
3.4.3 Other Constraints . . . 46
3.5 Software System Attributes . . . 46
3.5.1 Reliability . . . 46
3.5.2 Availability . . . 46
3.5.3 Security . . . 46
3.5.4 Maintainability . . . 46
3.5.5 Portability . . . 46

4 Formal Analysis Using Alloy . . . 47
4.1 Alloy Model . . . 47
4.2 Alloy Results . . . 54
4.2.1 Predicates . . . 54
4.2.2 Assertions . . . 68

5 Effort Spent . . . 73

6 References . . . 74
6.1 References . . . 74
6.2 Used Tools . . . 74

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

List of Figures

2.1 Domain Class Diagram. 13
2.2 Bike Path State Diagram. 14
3.1 User Use Cases. 20
3.2 Non-Registered User Use Cases. 20
3.3 Registered User Use Cases. 21
3.4 Use Case 1. 22
3.5 Use Case 2. 23
3.6 Use Case 3. 24
3.7 Use Case 4. 25
3.8 Use Case 5. 27
3.9 Use Case 6. 29
3.10 Use Case 7. 31
3.11 Use Case 8. 34
3.12 Use Case 9. 35
3.13 Use Case 10. 37
3.14 Use Case 11. 39
3.15 Use Case 12. 41
3.16 Use Case 13. 42

List of Tables

14 Requirements/Use Cases Mapping Table 43

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

1 Introduction

In recent years, cycling has become an increasingly popular and sustainable means of transportation. However, cyclists often face difficulties in finding accurate and up-to-date information about BPs. Existing data are usually scattered, incomplete, or not maintained, making it hard to evaluate the quality, safety, and usability of available routes. This lack of reliable information affects both occasional and regular cyclists, who may encounter poorly maintained paths, obstacles, or unsuitable routes without prior notice.

The **Best Bike Paths (BBP)** system is conceived to address these issues by providing a digital platform that collects, organizes, and displays data about BPs. Through manual input and automatic acquisition from mobile sensors, BBP builds a shared, up-to-date map of cycling routes enriched with statistics and environmental data, supporting users in discovering and maintaining the best paths for their journeys.

1.1 Purpose

Best Bike Paths (BBP) is a software designed to let users create and browse a catalog of BPs, that is a path in which there is an actual bike track or a street with few cars and a speed limit suitable for bikes.

For this purpose RUs can record their trips, storing various data like distance covered, average speed and, when available, also meteorological information retrieved from an XMS.

RUs can insert in the system information about the paths, like their status or the presence of potholes, whether using the manual mode, specifying the name of the streets composing the path with their relative status, or the automated mode, thanks to which users let BBP acquire data from the mobile device while biking. If willing to, the RUs can also publish the information entered.

More specifically, the automated mode can guess if the users are biking depending on their speed, it reconstructs the path with the GPS information and, thanks to the accelerometer and gyroscope technologies of the mobile device, it can also detect the presence of obstacles, like potholes. However, since this mechanism could produce false positives, the users will always have to confirm or correct the information acquired through this mode.

Finally, users, even non-registered ones, can submit an origin and a destination to BBP in order to visualize all paths between these two points that have been previously published by other RUs with all the information related. If more paths exist, they are shown based on their score, a parameter computed by the system based on the status of the path and how efficient it is in terms of letting the users reach their destination.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

1.1.1 Goals

From the description above, the following *goals* can be derived:

ID	Description
G1	RUs should be able to store information about their trips, like distance covered, average speed, and, if available, also meteorological information.
G2	RUs should be able to manually insert in BBP information about BPs, like status or potholes.
G3	RUs should be able to insert in BBP information about BPs, confirming or correcting what has been automatically retrieved by the BBP system.
G4	RUs should be able to publish the BPs they previously inserted.
G5	Any kind of user, even non-registered ones, by entering in BBP an origin and a destination, can visualize all the BPs between these two points previously published by other RUs, ordered based on their score.

1.2 Scope

1.2.1 World Phenomena

The following are the phenomena occurring in the *world* not observed by the machine:

ID	Description
WP1	A user takes a ride on a BP.
WP2	A user encounters obstacles in the BP.
WP3	A user rides at a certain average speed.
WP4	A user covers a certain distance on a trip.
WP5	An XMS retrieves meteorological information.
WP6	The GPS of a user collects information of a BP
WP7	The accelerometer and the gyroscope of a user device detects movements of the user on a BP

1.2.2 Shared Phenomena

The following phenomena involve both the *world* (the environment where users, paths and sensor exist) and the *machine* (the BBP software system).
They are grouped by controller (who initiates the phenomenon) and observer.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Controller: World - Observer: Machine

ID	Description
SP1	A user registers an account on BBP
SP2	A RU logs in BBP
SP3	A RU logs out of BBP
SP4	A user starts a manual insertion of a new BP
SP5	A user keeps track of its cycling activities
SP6	A user confirms information retrieved automatically by BBP
SP7	A user publishes information about their BPs
SP8	A user specifies the name of streets of a BPs
SP9	A user sets the status of a new BP
SP10	A user reports the presence of potholes in a BP
SP11	A user visualizes a BP between a given origin and destination
SP12	A user selects one of the suggested BPs to view its details
SP13	An XMS updates meteorological data
SP14	EGAS returns a validation response to BBP, verifying that the streets are connected and have speed limits below the threshold of 50 km/h.

Controller: Machine - Observer: World

ID	Description
SP15	BBP retrieves GPS coordinates from the user's device
SP16	BBP records speed data from device sensors
SP17	BBP records acceleration data from the device sensors
SP18	BBP records motion data from the device sensors
SP19	BBP shows trips statistics
SP20	BBP identifies potential potholes based on sensor data
SP21	BBP retrieves and displays weather data of the trip
SP22	BBP shows BPs on a map given origin and destination by user
SP23	BBP shows the BPs ordered by score based on condition, safety and effectiveness
SP24	BBP sends a street list or segment coordinates to EGAS for validation

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

1.3 Definition, Acronysm, Abbreviations - Glossary

Acronym	Definition
BBP	Best Bike Paths
BP	Bike Path
BPs	Bike Paths
Gn	Goal number n
WPn	World Phenomena number n
SPn	Shared Phenomena number n
RU	Registered User
RUs	Registered Users
XMS	External Meteorological Service
GPS	Global Positioning System
EGAS	External Geo-spatial Analysis System
EBP	Explorable Bike Path
DAn	Domain Assumption number n

1.4 Revision History

Version 1.0: initial release

1.5 Reference Documents

- The Requirement Engineering and Design Project specification document A.Y. 2025-2026.
- Slides of the Software Engineering 2 course

1.6 Document Structure

This document is organized into six main chapters, each of them describes a specific purpose in documenting the requirements and specifications of the RUs system.

1.6.1 Introduction

This chapter introduces the BBP system and its purpose. It defines the project goals, clarifies the system scope through the World and Machine model, identifies relevant phenomena (world and shared phenomena) and provides essential definitions and terminology in the Glossary. This chapter contextualizes the problem domain and sets the stage for the detailed analysis that follows.

1.6.2 Overall Description

This chapter provides a comprehensive view of the system through concrete scenarios that illustrate typical user interactions with BBP. It presents the domain model using UML class diagram to represent entities and their relationships, and state diagrams to capture dynamic behavior of key components. It also characterizes the domain in greater detail, summarizes the main product functions and documents all relevant domain assumptions and constraints that shape the system's boundaries.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

1.6.3 Specific Requirements

This chapter forms the core technical specification of the document. It details all external interface requirements, including user, hardware and software. The functional requirements are thoroughly documented through requirements, comprehensive use case descriptions with relative sequence diagrams. Additionally, this chapter specifies performance requirements, design constraints, and essential software system attributes such as reliability, availability, security, maintainability, and portability.

1.6.4 Formal Analysis Using Alloy

This chapter presents a formal verification of the system model using the Alloy specification language. It includes the complete Alloy model with signatures, facts, and assertions, demonstrates the model's correctness through example worlds, and validates key system properties and constraints. The formal analysis ensures logical consistency and helps identify potential ambiguities or contradictions in the requirements.

1.6.5 Effort Spent

This chapter provides transparency about the development process by documenting the time investment of each team member across different activities and deliverables.

1.6.6 References

This chapter lists all cited sources, standards, and reference materials used in preparing this document.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2 Overall Description

2.1 Product Perspective

2.1.1 Scenarios

Scenario 1: User Registers and logs in John, an experienced user who often rides his bike, decides to register into BBP after having heard of its usefulness. He opens the app on his mobile device and is welcomed with a login view. However he is not registered yet, so he taps on the registration button and enters his information, that is first and last name, his email and a new username and password associated with his account. After his account has been successfully created, he has been automatically logged in and he will be on the BBP home screen.

Scenario 2: User logs in While biking John broke his phone, so he promptly bought a new one and installed BBP on it. He is already a RU, so in order to use BBP he just needs to log in so, when welcomed with the log in view, he just enters his credentials (username and password) and successfully logs in.

Scenario 3: User records their trip A RU Amanda decides to record her next trip using a functionality of BBP. Before beginning the trip, she taps on the button “Record Trip” to enable BBP to gather information to provide some metrics about her ride and enters the name "My Trip". Amanda then starts biking and BBP, using the GPS functions of the mobile device on which it is installed, starts calculating the distance she is covering and, when Amanda stops after one hour and terminates the recording process, it also computes and shows the average speed and retrieves the meteorological information from the XMS it relies on.

Scenario 4: User explores their trips A RU Jane decides to take a look at the trips she previously recorded so, from the home screen, she taps on the "Your Trips" buttons and all the names of the trips she recorded are listed. She then taps on the trip "Trip12" and visualizes that she covered 10 km with an average speed of 20 km/h. She can also see that it was a sunny day, with a temperature of 26 C° and the speed of the wind was 5.4 km/h.

Scenario 5: RU enters information about a bike path manually A RU named Jacob decides to insert into his BBP app this new BP he found while riding alone on his bike in the countryside. When he finishes riding he enters in BBP on his mobile device and taps on the “Your Bike Paths” button, which leads to a list of all of the previous BPs he previously inserted, then taps on the “Add New Bike Path” button and selects the manual mode. He now enters the name he wants to give to his BP, the names of the streets with their city composing the path, that is “Via Armando Diaz Castelpoto” and “Via Aldo Moro Castelpoto”, reports a pothole he found in Via Aldo Moro and sets to “medium” the status of the BP. BBP uses an EGAS to verify that the streets are connected and their speed limit is below 50 km/h, which is successfully done. After finishing this process BBP navigate Jacob onto the view of the list of his BPs and now he can see the BP he entered. He taps on it and can see the information he previously entered.

Scenario 6: RU enters information about a bike path automatically RU Andrew enters in BBP, taps on the “Your Bike Paths” button and then taps on the “Add New Bike Path” selecting the automated mode, in order to automatically insert into his BBP app the new BP he is about to explore. As soon as he enables the automated mode and starts biking, BBP starts recording the path he is traversing using the GPS technology of his device. Suddenly Andrews stops because he receives a call from his mother to which he answers and BBP, thanks to the GPS, understands Andrew is not biking anymore because of his speed being zero. After finishing the call he starts biking again and runs onto a pothole he did not see soon enough, causing a sudden movement detected by the accelerometer and the gyroscope that lets BBP understand that a pothole is located in that precise point of the BP. After 20 minutes, Andrew

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

stops and terminates the recording of the path on BBP, that asks him if the potholes it detected through the trip (one in this case) were effectively there and if the automatically computed status of “medium” is coherent. The potholes are defined through the distance from the beginning of the path in kilometers. Andrew confirms all the information, sets a name to the BP and successfully inserts his BP into his BBP app.

Scenario 7: RU publishes a BP they previously inserted RU Jessica decides to publish a BP she named “Path 10” that she previously inserted using the automated mode. In order to do that she taps on the “Your Bike Paths” button and then taps on the “Path 10” BP showing all the related info. In the current view she taps on the “Publish Bike Path” button to publish her BP, so BBP checks if such a BP has already been published by somebody else. BBP finds out that the exact same path has already been published by other users so it doesn’t create a new instance of a BP in its database but it just adds to the already existing BP another version of it associated with Jessica with all the information she entered. She is then navigated to the view of the BP that she just published and she can see that there is also her version of the BP, on which she can tap and see all the related information.

Scenario 8: Unregistered user looks for a BP Marco installs the BBP app, opens it and is welcomed with the login view, but he decides not to register, so he taps on the “Enter as guest” button. As a guest he taps on the “Explore Bike Paths” button (the only thing he can do) and he is asked to enter an origin and a destination of the BP he wants to find. He enters “Via Porpora, Milano” as origin and “Via Golgi, Milano” as destination and BBP finds multiple paths between these two points. BBP then queries the EGAS to return the map corresponding to the BP with the best score and then displays it to the user. On the side, it shows a list of all the other BPs between these two points, ordered in descending order of score. Marco taps on the third BP of the list, which is now visualized on the map (with the same interaction with the EGAS as before) instead of the previous one. Since he particularly likes this one, he decides it will be the BP of his next trip.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2.1.2 Domain Class Diagram

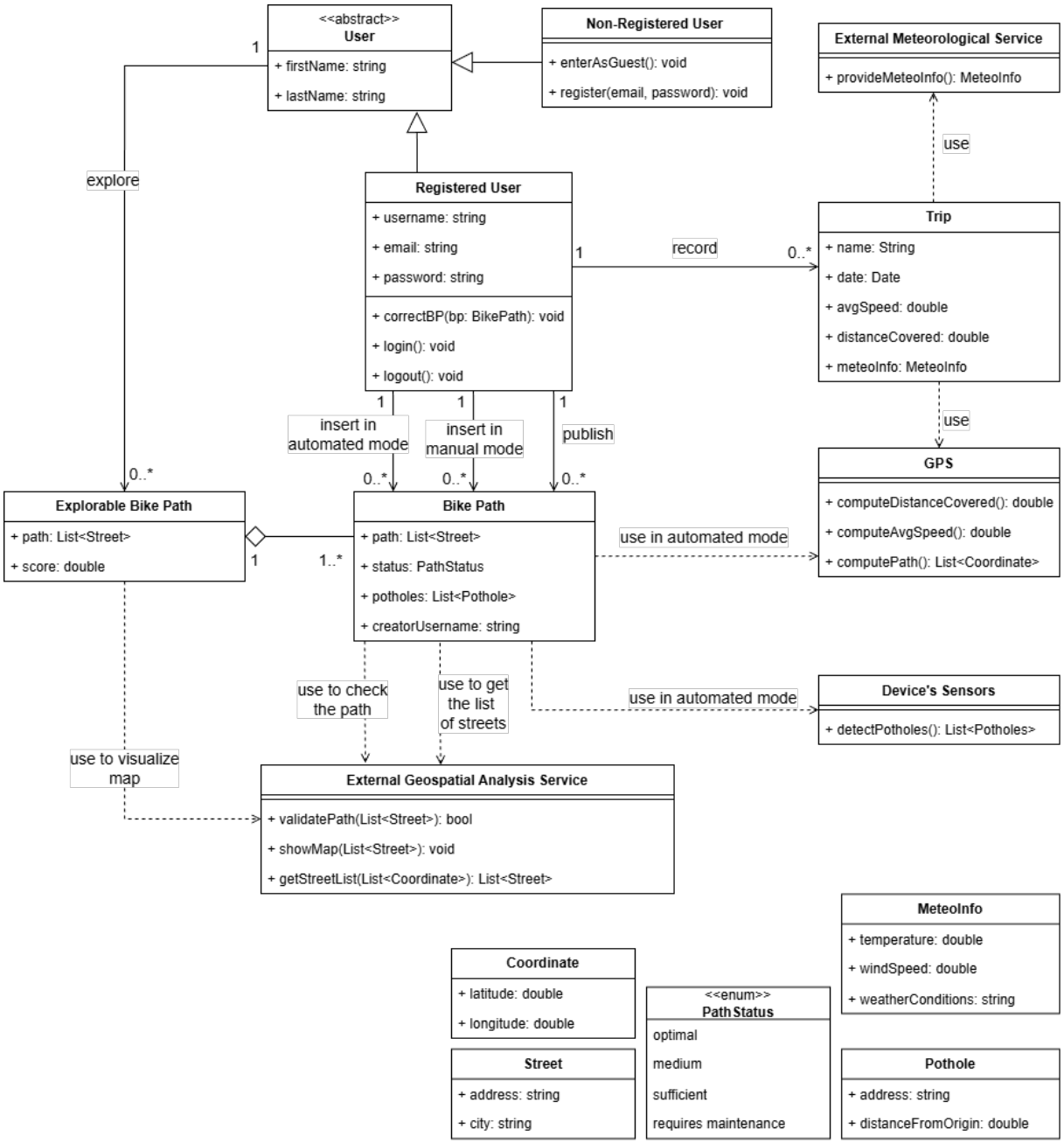


Figura 2.1: Domain Class Diagram.

The class diagram in figure 2.1 shows the main entities of the world. Users can be of two types: RUs or non-registered users. Non-registered users can only explore public BPs and register, while RUs can use all BBP's functionalities and also have a username and a password.
A BP is associated to one and only RU, however, if more RUs publish a BP composed by the same streets, they will all collapse in an EBP, that is what is actually shown when a user uses the "Explore Bike Paths" function when looking for a BP. An EBP has a score computed by the system, based on an average of all the statuses of the BPs associated to it, and the list of streets composing it. So an EBP is composed by at least a published BP and the information related to its status is stored in the different BPs composing it.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

RUs can insert BPs in manual mode, so they manually enter the streets composing the BP, the potholes and the status. A pothole is characterized by the address of the street it is found in and the distance in kilometers from the origin of the BP, while the status is a general evaluation of the BP that can have only four different values: "optimal", "medium", "sufficient" and "requires maintenance".
If a BP is inserted in manual mode, an EGAS is needed to check if all the streets entered by the RU are existing, have a suitable speed limit and are one next to each other. BBP relies on the EGAS also when a user looks for EBPs using the "Explore Bike Paths" function and a map with the BP found needs to be displayed.
RUs can also decide to insert a BP they traverse using the automated mode, a functionality that can be enabled before starting the trip that automatically builds the information that the user would otherwise had to insert manually, that is the streets, the potholes and the status. In order to do so, BBP relies on the device's sensors, referring to both the phone accelerometer and gyroscope, to detect potholes and on the phone GPS to detect which streets compose the BP. The system may detect some non-existing potholes, so the user, before confirming the insertion, may correct some false positives. However, the GPS can just build a list of coordinates (latitude, longitude), so BBP relies on the EGAS to convert the list of coordinates into the list of the corresponding streets.
RUs can also record their trips, so BBP again relies on the phone's GPS to compute the distance covered and the average speed, and on an XMS, if available, to retrieve meteorological information, so the weather conditions, the temperature and the wind speed.

2.1.3 State Diagrams

Here follows a state diagram describing the different states describing the evolution of BPs from their creation in the two different modes, to their publication as described in the section above.

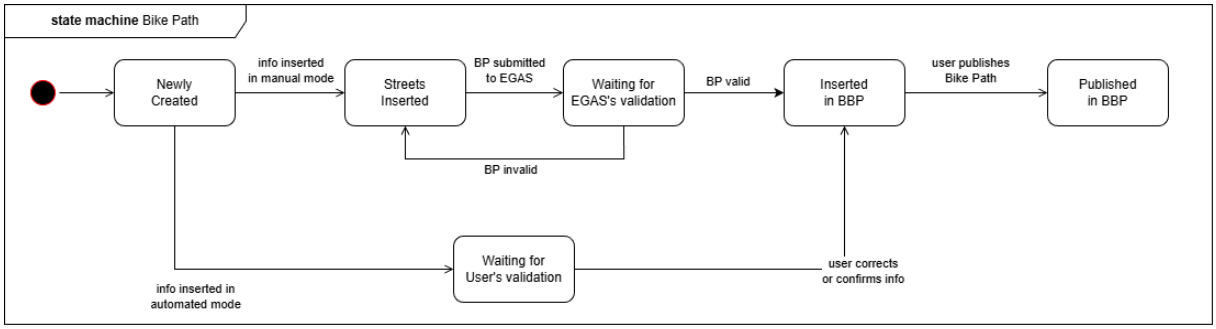


Figura 2.2: Bike Path State Diagram.

2.2 Product Functions

In this section, the most important functionalities of BBP are presented and described in a broad way, in order to offer a deeper description, comprehensive of use cases and use case diagrams, in section 3.

General Functionalities

- **User Registration**: BBP allows non-registered users to register in the system
- **User Log In**: BBP allows RUs to log in the system
- **Entering as Guests**: BBP allows non-registered users to enter as guests on the platform

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Trip Functionalities

- **Trip Recording**: when RUs tap the "Record Trip" button, BBP allows them to record their trips, that is, collecting the following information about each of their trips:

  – average speed

  – distance covered

  – meteorological information retrieved by the XMS if available

  When the XMS is not available, BBP will simply report N/A next to the "Meteorological Information" writing. When the recording is stopped, BBP also sets the date associated with the Trip to the current date. Users must then give a name to the trip they just recorded to correctly store it. The GPS is also able to understand if a user is traveling by bike, in fact, if BBP detects a speed over 60 km/h, it infers that the user is traveling on another means of transport (a car for example) and cancels the recording

- **Trip Navigation**: when RUs tap the "Your Trips" button from the home screen, BBP lists all the trips they previously recorded. When the user taps on one of them, BBP displays the information related to it (speed, distance, meteorological information)

Bike Path Functionalities

- **Manual Mode**: BBP allows RUs to manually insert information about a BP they traversed by letting them enter the following data:

  – name he wants to give to the BP (private)

  – list of streets composing the BP

  – list of potholes they encountered while biking

  – status

  An EGAS then verifies that all the streets entered have a speed limit below 50 km/h and that they actually compose a BP (they are connected to each other). If the EGAS gives a negative response, the RU must correct the information about the streets

- **Automated Mode**: BBP allows RUs to automatically insert information about a BP while they are traversing it. In fact, while biking, BBP uses the device's sensors it is installed on to detect potholes and the GPS to build the sequence of coordinates composing the BP. These coordinates are then sent to the EGAS, that is able to reconstruct from them the streets of the BP. Like in the trip functionality, the GPS is also able to understand if a user is traveling by bike, so, if BBP detects a speed over 60 km/h, it infers that the user is traveling on another means of transport (a car for example) and cancels the recording. In the end, the RUs stop the recording, give a name to the BP and confirm or correct the information about the potholes detected and the score automatically computed by BBP on the basis of how many potholes were encountered

- **Bike Paths Navigation**: when RUs tap the "Your Bike Paths" button from the home page, BBP lists all the BPs they previously inserted. When the user taps on one of them, BBP displays the information related to it

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Public Bike Path Functions

- **Publishing Bike Paths**: BBP allows RUs to publish the BP they previously inserted using either automated or manual mode. If there already exists an EBP with the same exact streets as the one the user is now publishing, the BP is included in the corresponding EBP, while, if no EBPs with such a path exist, then a new EBP is created and it is associated only to the BP just published
- **Exploring Public Bike Paths**: BBP allows both non-registered users and RUs to search for BPs previously published by other RUs. By entering the origin and destination streets of the BP they are looking for, users can visualize all the different EBPs between these two points ordered in descending order of their score. In particular, there is a map, that initially shows the EBP with the highest rating, and a scrollable list of all the other ordered EBPs. When tapping on one of the EBPs in the list, the corresponding BP is shown on the map and, by tapping again on it in the list, BBP displays all the different BPs associated to the EBP with all the information related. Each time a new map is shown, BBP first queries the EGAS to generate a map with the given EBP, and then displays it. When the user taps on an EBP twice to see all the different BPs' information, if the user taps on one of the BPs shown, the map is updated with the potholes that the RU associated with the BP selected has reported.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2.3 User Characteristics

BBP is designed to be used by users and they can either be registered or non-registered users:

Non-registered Users: they can only register in BBP and become RUs or enter as guests and explore EBPs published by other users.

Registered Users: they can explore EBPs like non-registered user and, in addition to that, they can:

- record trips
- explore previously recorded trips
- insert BPs in manual mode
- insert BPs in automated mode
- explore previously inserted BPs
- publish previously inserted BPs

2.4 Assumptions, Dependencies and Constraints

In this last section there is a list of the domain assumptions, that is a list of all the facts about the world we assume to be true.

ID	Description
DA1	Users enter their first and last name correctly when registering
DA2	Users correctly report the presence of a pothole when they encounter it in manual mode
DA3	Users evaluate BPs by entering an appropriate status when entering inputs in manual mode
DA4	Users confirm information retrieved by BBP through the automated mode only when it is actually correct
DA5	Users correct information retrieved by BBP through the automated mode only when it is wrong and modify it in a correct way
DA6	The device's sensors retrieve data in a correct way
DA7	The GPS retrieve data in a correct way
DA8	The XMS, if it is available, return correct meteorological information
DA9	The EGAS correctly retrieves information about BPs

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3 Specific Requirements

3.1 External Interface Requirements

This chapter describes the interfaces between the BBP system and external entities, including hardware devices, software systems and communication channels. The purpose is to specify the types of interactions the system must support.

3.1.1 User Interfaces

BBP provides a graphical user interface through a mobile client application, installable on smartphones. The interface allows the user to submit information, visualize system output and interact with the functionalities defined in the requirements.

3.1.2 Hardware Interfaces

BBP is intended to run on mobile devices. No external hardware is required; however, some functionalities rely on built-in mobile sensors such as GPS, accelerometer, and gyroscope, when available on the user's device. If sensor access is denied or sensors are unavailable, the corresponding functionalities cannot be performed.

3.1.3 Software Interfaces

The BBP system interacts with two external software services through the backend:

- XMS: to retrieve meteorological information associated with recorded trips.
- EGAS: to validate manually entered BPs, to reconstruct BPs by map-matching GPS traces into street lists and to return visualizable data of maps.

The mobile client relies on the mobile operating system services/APIs to access sensor data and to manage runtime permissions. All backend-to-service interactions occur through standard API calls.

3.1.4 Communication Interfaces

All communication between the mobile client and the BBP backend occurs through secure network protocols, using a RESTful interface. Communication between the BBP backend and external services (XMS, EGAS) occurs through their exposed APIs. A network connection is required for all operations that involve synchronization with the backend, map retrieval, or external service queries, while sensor acquisition is performed locally on the device through OS APIs.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.2 Functional Requirements

3.2.1 Functional Requirements List

ID	Functional Requirement
R1	The system shall allow a user to register an account.
R2	The system shall allow a RU to log in.
R3	The system shall allow a RU to log out.
R4	The system shall allow a user to enter as a guest.
R5	The system shall allow a RU to record a trip.
R6	The system shall acquire GPS data during a trip.
R7	The system shall record the distance covered during a recorded trip.
R8	The system shall record the average speed of a recorded trip.
R9	When meteorological information is available from the XMS, the system shall retrieve the corresponding weather data for the recorded trip.
R10	The system shall allow a RU to view previously recorded trips.
R11	The system shall allow a RU to insert a BP manually.
R12	When a RU submits a manually entered BP, the system shall validate the path through the EGAS.
R13	The system shall allow a RU to report potholes within a manually inserted BP.
R14	The system shall allow a RU to insert a BP automatically.
R15	When GPS coordinates are acquired during automated insertion, the system shall reconstruct the BP from the recorded positions.
R16	When significant movements are detected through motion sensors, the system shall infer the presence of potential potholes.
R17	When the system detects information automatically during BP insertion, the system shall allow the user to confirm or correct the detected data.
R18	When GPS data indicate that the user's speed is 0, the BBP system shall pause the automated BP insertion or the trip insertion.
R19	When GPS data indicate that the user's speed exceeds 60 km/h the BBP system shall terminate the automated BP insertion or the trip insertion.
R20	The system shall allow a RU to assign a status to a BP.
R21	The system shall allow a RU to publish a BP they inserted.
R22	The system shall allow any user to request EBP between an origin and a destination.
R23	When the user specifies an origin and a destination, the BBP system shall retrieve all EBPs connecting them.
R24	When the user specifies an origin and a destination, the BBP system shall obtain the corresponding map of the highest-scored EBP from the EGAS and display it.
R25	The system shall order BP in the EBPs list according to the score.
R26	When an EBP is selected from the list, the system shall request from the EGAS the map for the selected EBP and display it together with its detailed information.
R27	The system shall allow a RU to view the BPs they previously inserted.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.2.2 Use Cases Diagrams

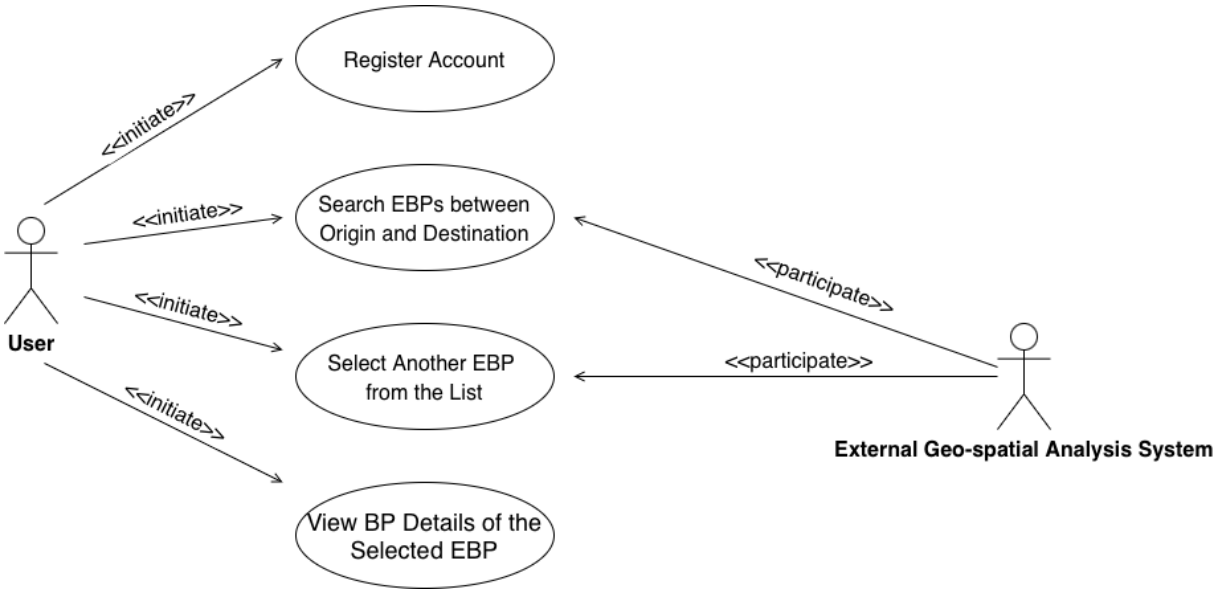


Figura 3.1: User Use Cases.

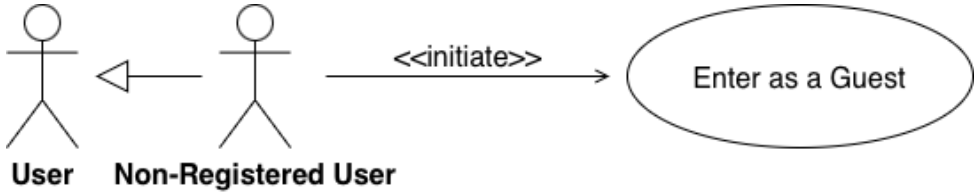


Figura 3.2: Non-Registered User Use Cases.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

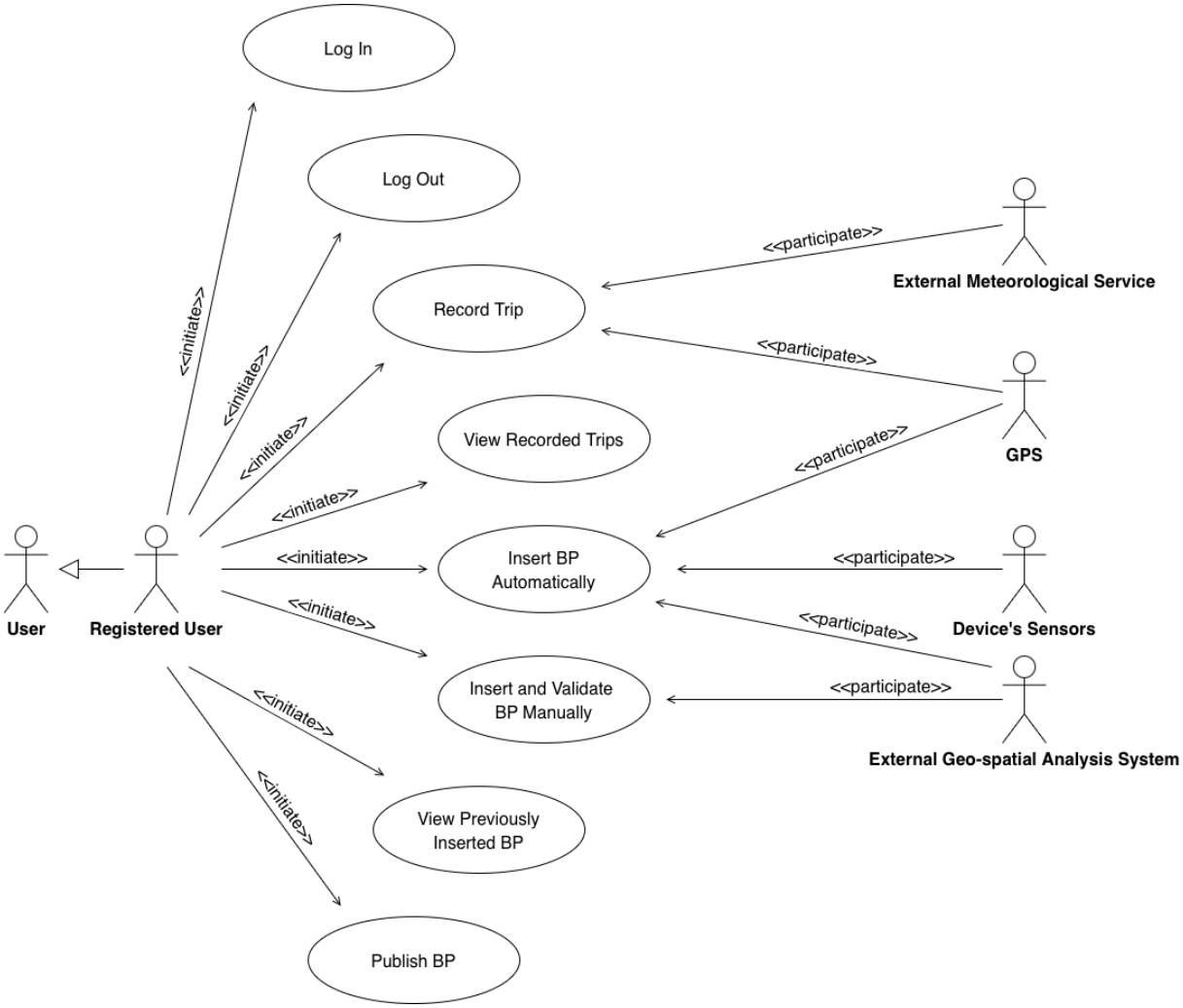


Figura 3.3: Registered User Use Cases.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.2.3 Use Cases

[UC1] - Register Account

Actor	User
Entry condition	User is not authenticated and is on the registration tab
Event Flow	1. The user fills out a form entering the required personal information (first name, last name, username, email, password). 2. The BBP system validates the inserted data. 3. The BBP system checks that the email and username are not already in use. 4. The BBP system creates the new account. 5. The BBP system authenticates the user and takes them to the home screen.
Exit condition	A new registered account is created and the user is authenticated.
Exceptions	• Missing or invalid data inserted. • Email or username already in use. • Password does not meet the validation rules.

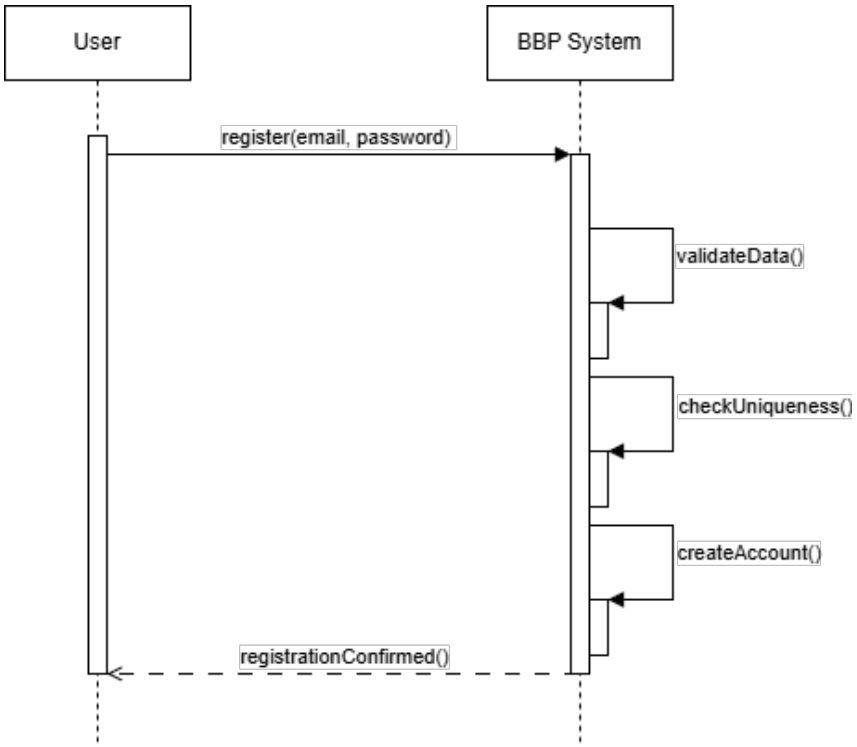


Figura 3.4: Use Case 1.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC2] - Log In

Actor	RU
Entry condition	The RU is on the login view.
Event Flow	1. The RU enters username and password. 2. The BBP system validates the credentials. 3. The BBP system authenticates the RU and navigate them to the home screen.
Exit condition	The RU is authenticated and logged in.
Exceptions	• Invalid username and password inserted. • Missing data in one or both fields.

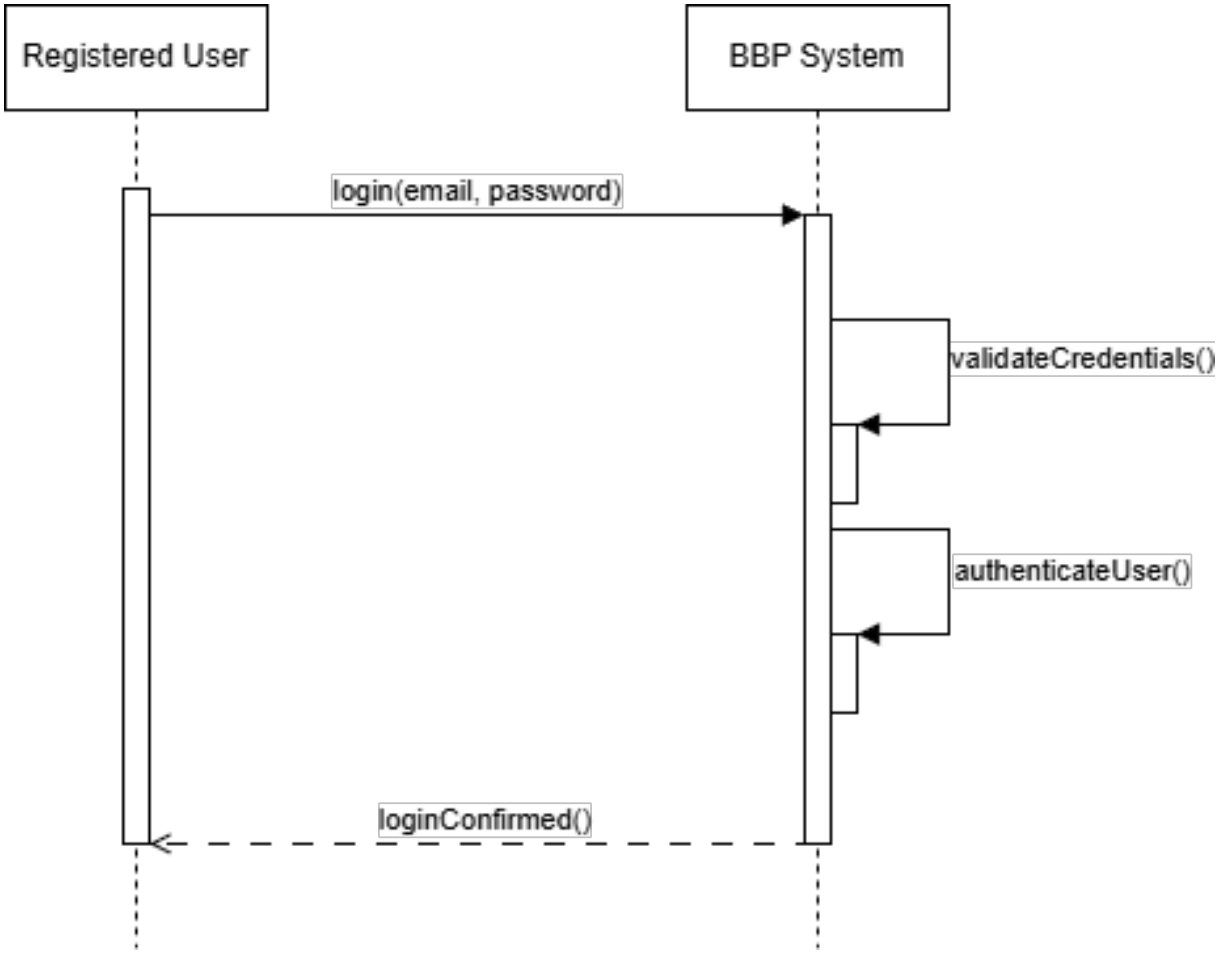


Figura 3.5: Use Case 2.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC3] - Log Out

Actor	RU
Entry condition	The RU is authenticated and is on any screen of the BBP mobile application
Event Flow	1. The RU selects the "Log Out" option. 2. The BBP system performs the log out operations. 3. The BBP system navigate the user to the login screen.
Exit condition	The RU is no longer authenticated
Exceptions	• The BBP system is unable to complete the log out operations due to an internal error

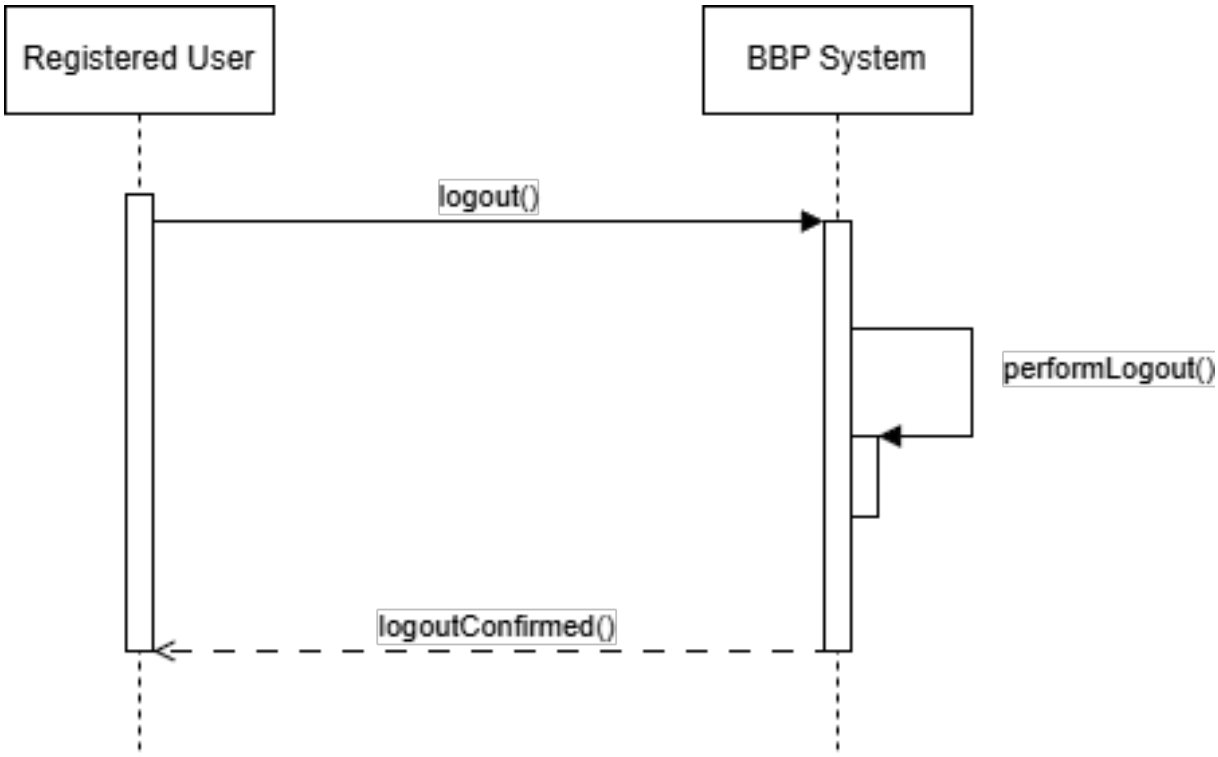


Figura 3.6: Use Case 3.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC4] - Enter as Guest

Actor	Non-registered user
Entry condition	The user is not authenticated and is on the login view.
Event Flow	1. The non-registered user selects the "Enter as a Guest" option. 2. The BBP system grants a Guest access to the BBP mobile application. 3. The BBP system navigates the non-registered user to the Explore BPs for non-registered user screen.
Exit condition	The user accesses the BBP system in Guest mode
Exceptions	• The BBP system is unable to load public EBPs due to an internal error. • The BBP system is unavailable due to an internal error.

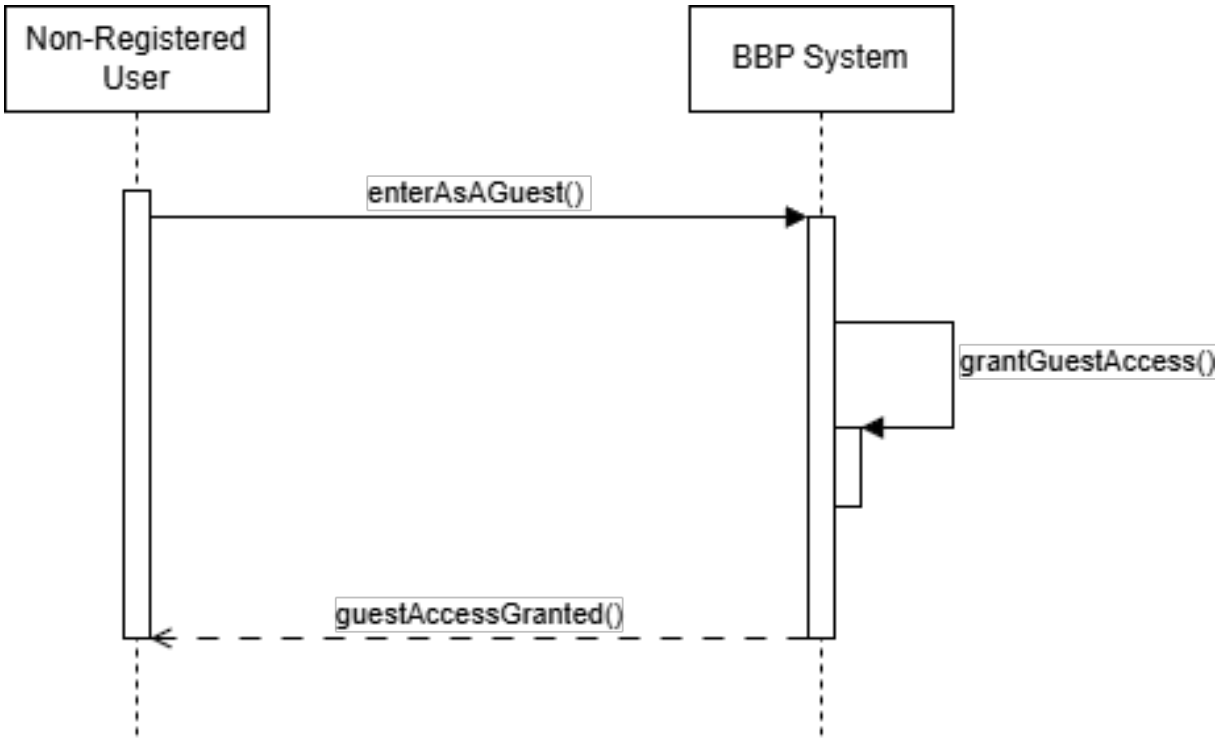


Figura 3.7: Use Case 4.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC5] - Record Trip

Actor	RU, GPS, XMS
Entry condition	The RU is authenticated and has opened the trip recording view
Event Flow	1. The RU starts a new trip recording. 2. The BBP system begins collecting GPS positions. 3. The BBP system continuously updates the distance covered. 4. The BBP system continuously updates the average speed. 5. The BBP system uses GPS speed information to determine if the user is biking: • If the detected speed is 0 km/h, then the BBP system pauses the trip recording. • If the detected speed exceeds 60 km/h, then the BBP system stops the trip recording. 6. The BBP system retrieves weather data from the XMS. 7. The RU stops the recording. 8. The BBP system finalizes the trip and makes it available for later actions.
Exit condition	The recorded trip, including distance, speed, and weather data, is stored and visible to the RU.
Exceptions	• GPS data unavailable. • The XMS is unavailable (weather data are not retrieved).

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

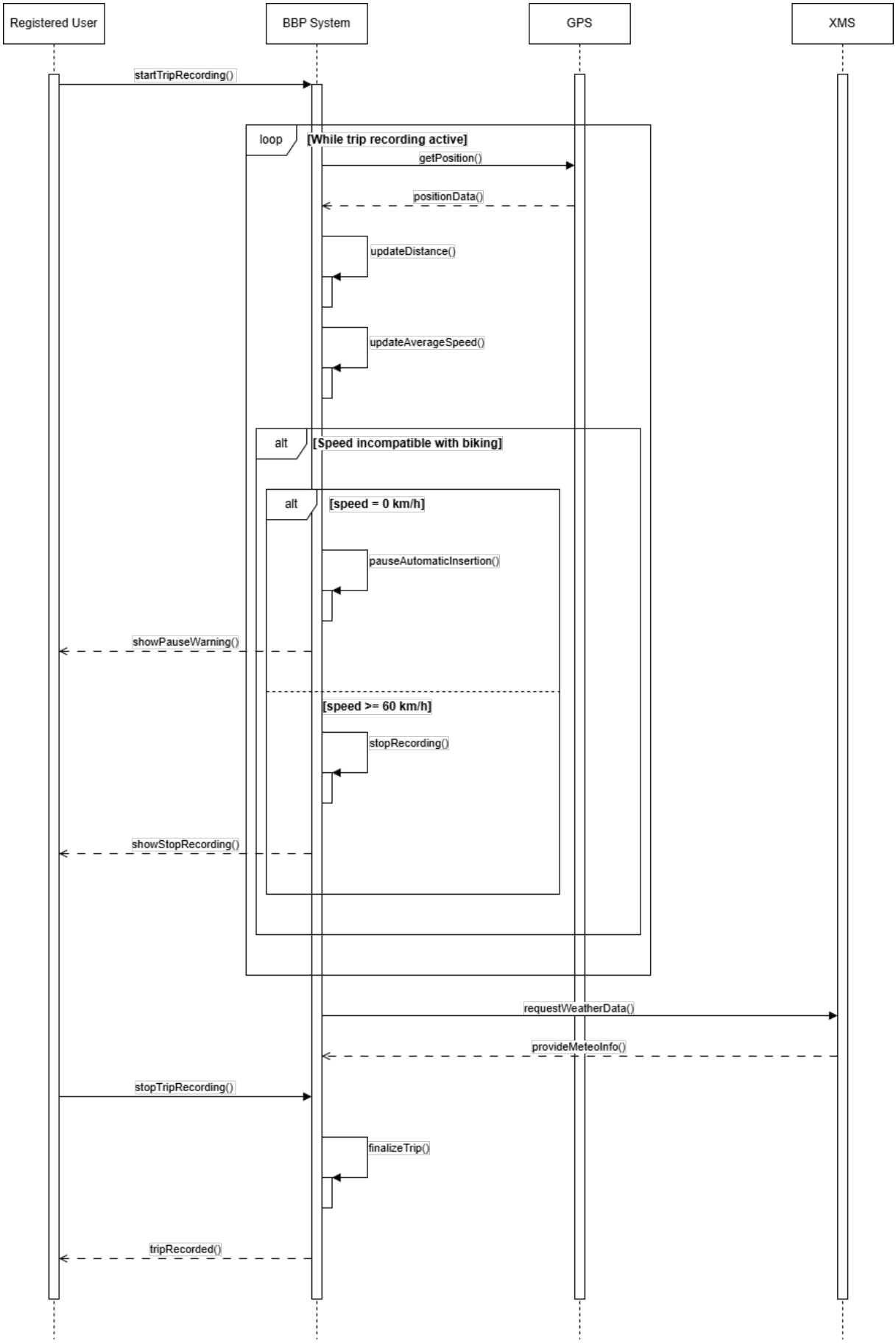


Figura 3.8: Use Case 5.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC6] - View Recorded Trips

Actor	RU
Entry condition	The RU is authenticated and has opened the "Recorded Trip" section
Event Flow	1. The RU requests access to the list of recorded trips. 2. The BBP system retrieves the RU's previously recorded trips. 3. The BBP system displays the list of trips. 4. The RU selects a specific trip to view its details. 5. The BBP system displays the information related to the selected trip.
Exit condition	The RU views the details of one of their recorded trips.
Exceptions	• The RU has no recorded trips (empty list) • The BBP system is unable to retrieve trip data due to an internal error.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

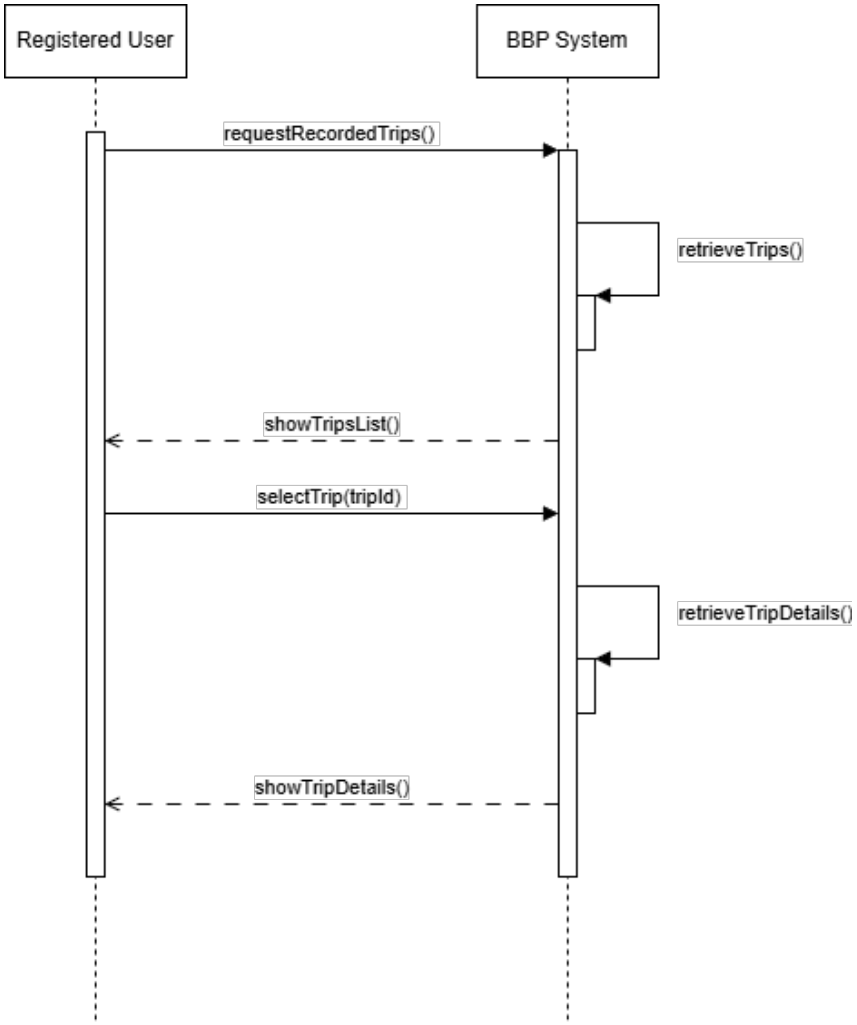


Figura 3.9: Use Case 6.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC7] - Insert and Validate BP Manually

Actor	RU, EGAS
Entry condition	The RU is authenticated and has opened the manual BBP insertion view.
Event Flow	1. The RU selects the streets composing the BP. 2. The RU assigns a status to the BP. 3. The RU optionally reports potholes encountered along the BP. 4. The BBP system checks that all required fields have been provided. 5. The BBP system sends the manually inserted BP to the EGAS for validation. 6. The EGAS checks the validity of the sequence of streets. 7. The EGAS returns the validation result to the BBP system. 8. The BBP system informs the RU about the validation outcome. 9. If the BBP has been validated, the BP has been successfully inserted into BBP; otherwise, the RU can correct the information entered or end the process.
Exit condition	The RU is informed of the validation result and the valid BPs are stored, invalid BPs are discarded.
Exceptions	• Missing required fields. • Invalid format or unacceptable values in the inserted data. • The EGAS is unavailable due to an error. • The EGAS returns an invalid or incomplete response. • The BP is invalid according to EGAS validation.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

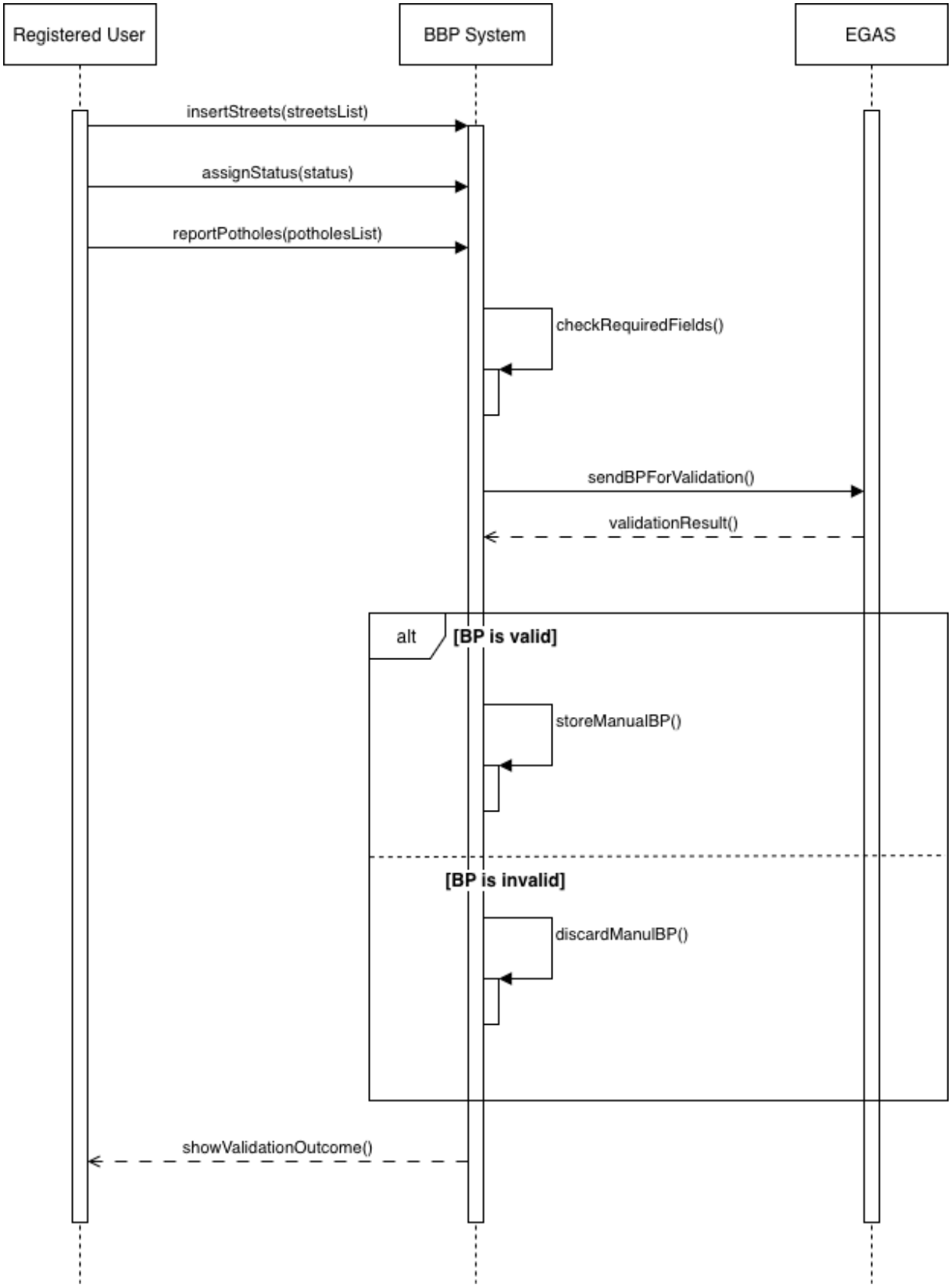


Figura 3.10: Use Case 7.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC8] - Insert BP Automatically

Actor	RU, GPS, Device's Sensors, EGAS
Entry condition	The RU is authenticated and is on the automatic BP insertion screen.
Event Flow	1. The RU starts the automatic BP insertion mode. 2. The BBP system starts collecting GPS data and, when permitted, motion sensor data. 3. While the automatic mode is active, the BBP system continuously updates the GPS trace and computes the current speed: • If the detected speed is 0 km/h, the BBP system pauses the automated BP insertion. • If the detected speed exceeds 60 km/h, the BBP system stops the automated BP insertion. 4. When significant movements are detected through sensors, the BBP system marks potential potholes/anomalies and associates them with the current position along the trace. 5. The RU stops the automatic BP insertion. 6. The BBP system sends the collected GPS trace to the EGAS to obtain the corresponding street list (map-matching). 7. The EGAS returns the reconstructed BP street list to the BBP system. 8. The BBP system prepares the automatically detected BP data using the information returned by EGAS. 9. The BBP system displays the reconstructed BP on a map, together with the detected potholes, to the RU. 10. The RU either confirms the detected data or corrects them. 11. The RU finalizes the BP data. 12. The BBP system stores the confirmed BP data and notifies the RU of the successful insertion.
Exit condition	The confirmed BP data are stored by the BBP system and are available for subsequent actions. The insertion may have been paused automatically when speed is 0 km/h, or stopped automatically when speed exceeds 60 km/h.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Exceptions	• GPS data are unavailable due to an error or missing permission. • Sensor data are unavailable due to an error or missing permission. • The EGAS is unavailable due to an error. • The EGAS returns an invalid or incomplete response, so the BP cannot be correctly reconstructed. • The confirmed BP data cannot be stored due to an internal error.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

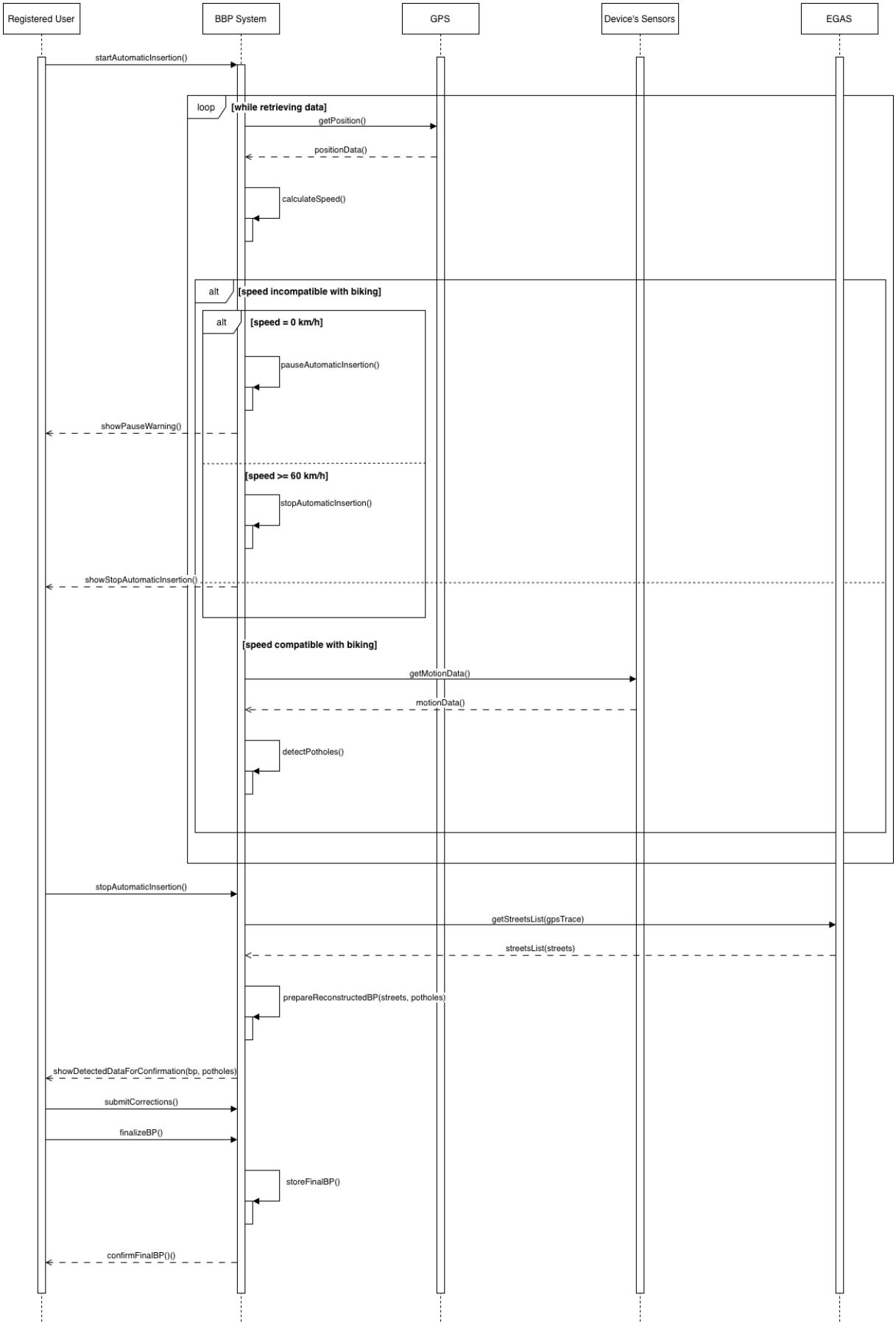


Figura 3.11: Use Case 8.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC9] - View previously inserted BPs

Actor	RU
Entry condition	The RU is authenticated and has opened "Your BPs" section.
Event Flow	1. The RU requests access to the list of BPs they previously inserted. 2. The BBP system retrieves the RU's previously inserted BPs. 3. The BBP system displays the list of BPs, showing for each BP its main information. 4. The RU selects a specific BP to view its details. 5. The BBP system displays the detailed information related to the selected BP.
Exit condition	The RU views the details of one of their previously inserted BPs.
Exceptions	• The RU has no previously inserted BPs, so it is shown an empty list. • the BBP system is unable to retrieve BP data due to an internal error.

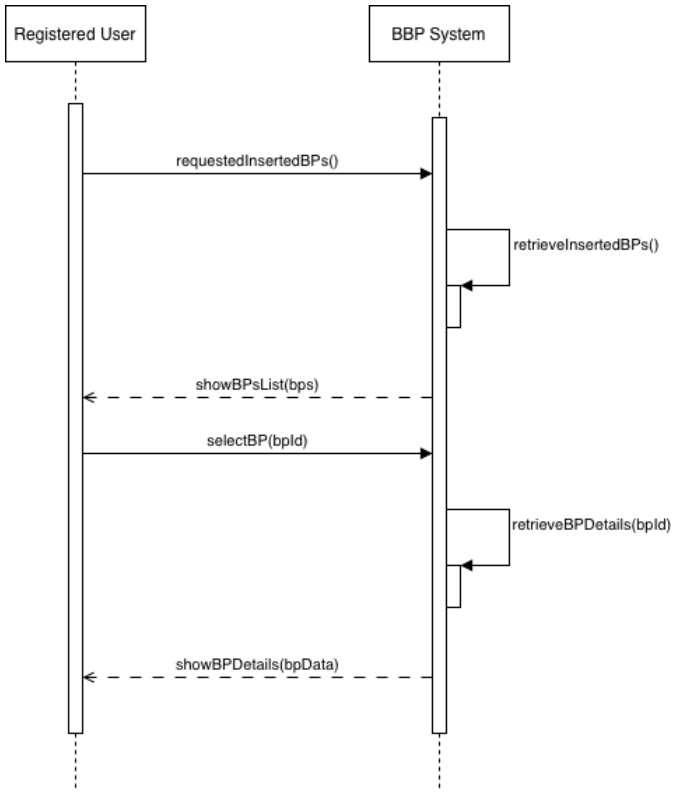


Figura 3.12: Use Case 9.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC10] - Publish BP

Actor	RU
Entry condition	The RU has inserted a BP (manually or automatically) and confirmed all required information.
Event Flow	1. The RU chooses to publish the BP. 2. The BBP system checks whether an EBP with the same exact streets already exists among the published EBPs. 3. If no such EBP exists, the BBP system creates a new EBP and associates it with the BP. 4. If such an EBP exists, the BBP system includes the BP in the corresponding EBP. 5. The BBP system confirms to the RU that the BP has been made publicly available.
Exit condition	The BP becomes publicly available within the system.
Exceptions	• Missing required information to complete publication. • The BBP system is unable to publish the BP due to an internal error.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

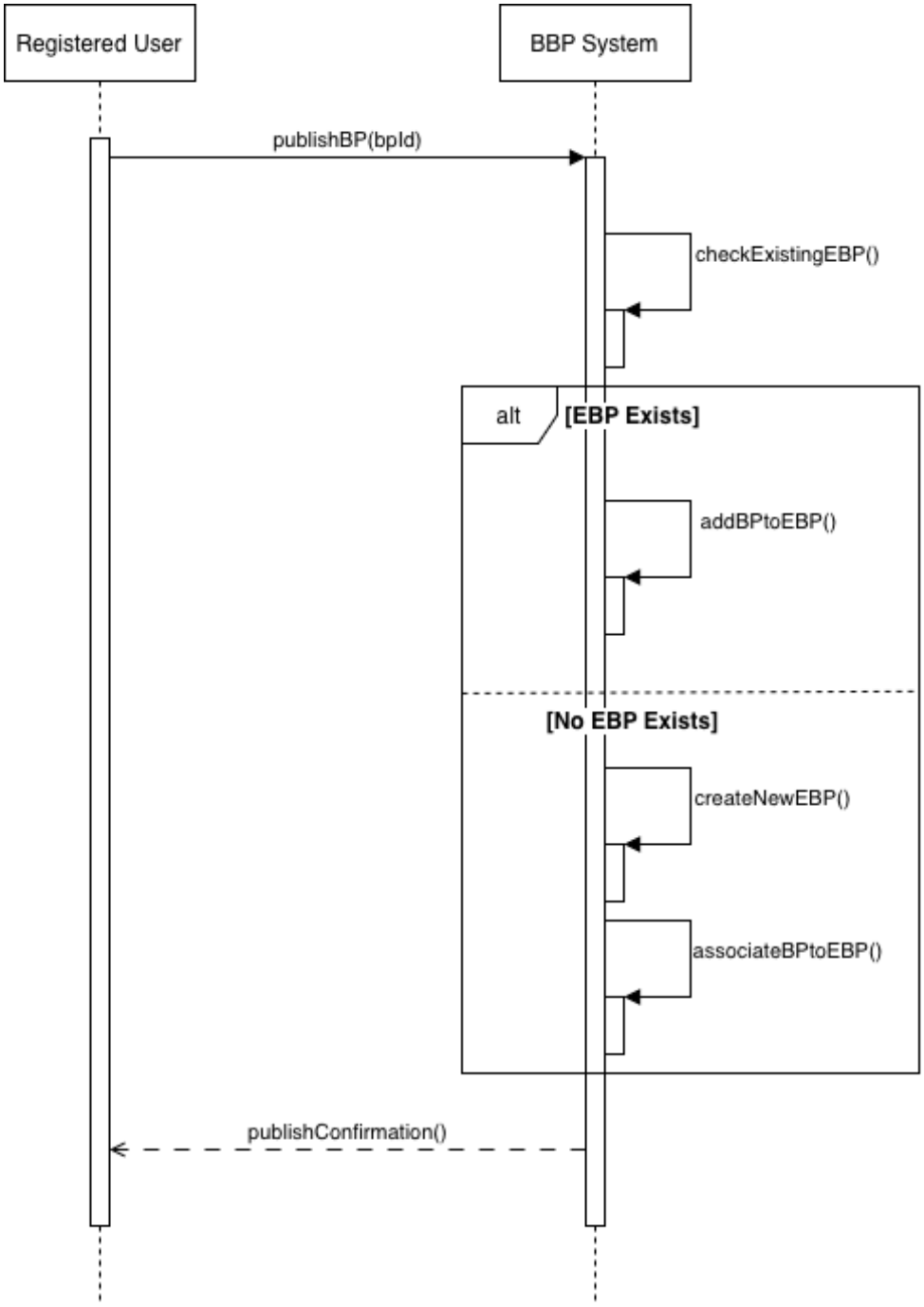


Figura 3.13: Use Case 10.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC11] - Search EBPs between Origin and Destination

Actor	User, EGAS
Entry condition	The user is on the EBP searching view.
Event Flow	1. The user inserts an origin and a destination. 2. The BBP system retrieves all EBPs connecting the specified origin and destination. 3. The BBP system orders the EBPs according to their score. 4. The BBP system requests the map of the EBP with the highest score from the EGAS. 5. The EGAS returns the map of the EBP with the highest score. 6. The BBP system displays the map. 7. The BBP system displays the ordered list of retrieved EBPs in descending order of score.
Exit condition	The user sees the highest scored EBP on the map provided by EGAS and the ordered list of EBPs is available for selection.
Exceptions	• Invalid origin or destination provided. • No EBPs available between the selected origin and destination. • The BBP system is unable to retrieve EBPs due to an internal error. • The EGAS is unavailable due to an error. • The EGAS returns an invalid or incomplete response.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

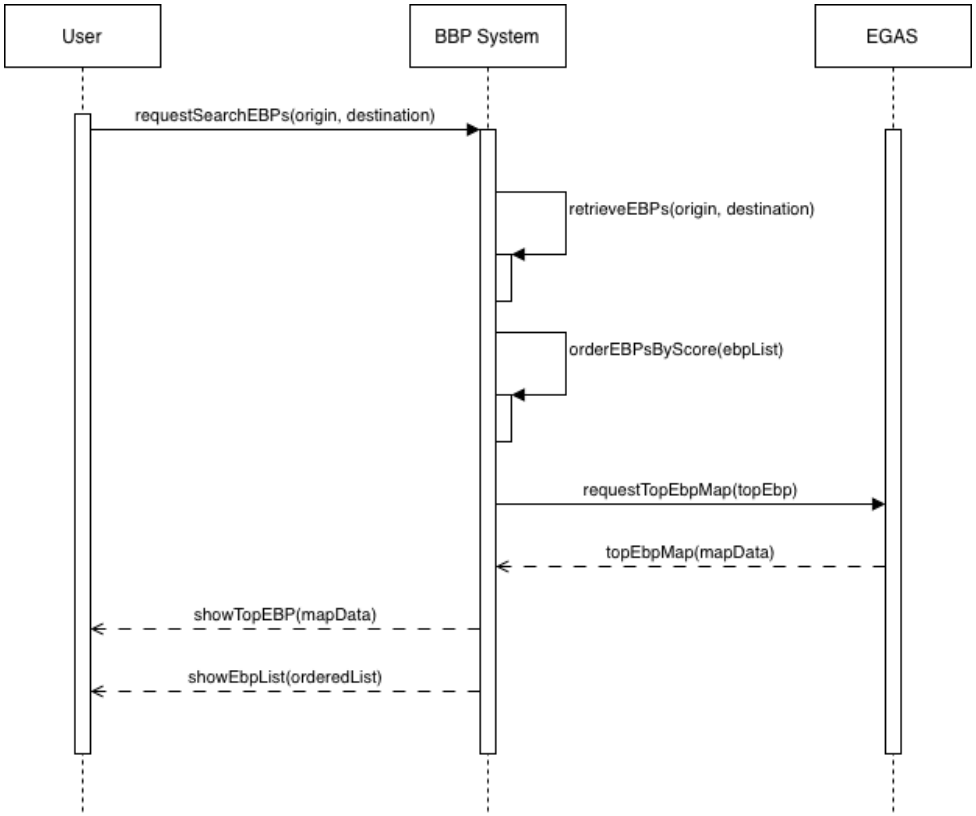


Figura 3.14: Use Case 11.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC12] - Select Another EBP from the List

Actor	User, EGAS
Entry condition	The list of EBPs and the highest-scored EBP on the map are displayed to the user.
Event Flow	1. The user selects an EBP from the list. 2. The BBP system retrieves the selected EBP's information. 3. The BBP system requests from the EGAS the map of the selected EBP. 4. The EGAS returns the map of the selected EBP. 5. The BBP system displays the map provided by EGAS on the screen.
Exit condition	The selected EBP is displayed to the user on the map provided by EGAS.
Exceptions	• The selected EBP cannot be retrieved due to an internal error. • The map cannot be updated due to an internal error. • The EGAS is unavailable due to an error. • The EGAS returns an invalid or incomplete response.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

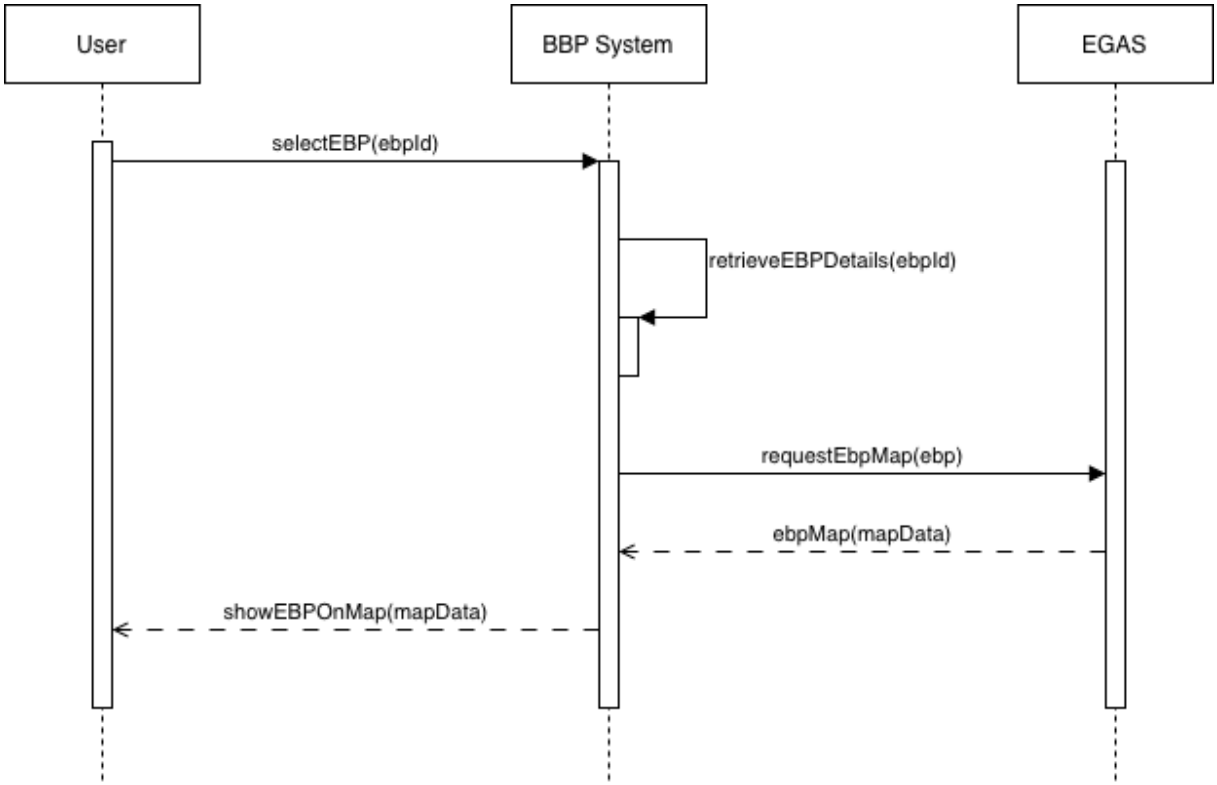


Figura 3.15: Use Case 12.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[UC13] - View BP Details of the Selected EBP

Actor	User
Entry condition	The user has selected an EBP from the list and requests to view its associated BPs.
Event Flow	1. The BBP system retrieves all BPs associated with the selected EBP. 2. The BBP system displays the list of BPs composing the EBP, including their status and pothole information. 3. The user selects one BP from the list to view its details. 4. The BBP system displays the map highlighting the potholes detected by the RU associated with the BP selected and the detailed information of the selected BP.
Exit condition	The user visualizes the map and the detailed information of the selected BP.
Exceptions	• The BBP system cannot retrieve the BPs of the selected EBP due to an internal error.

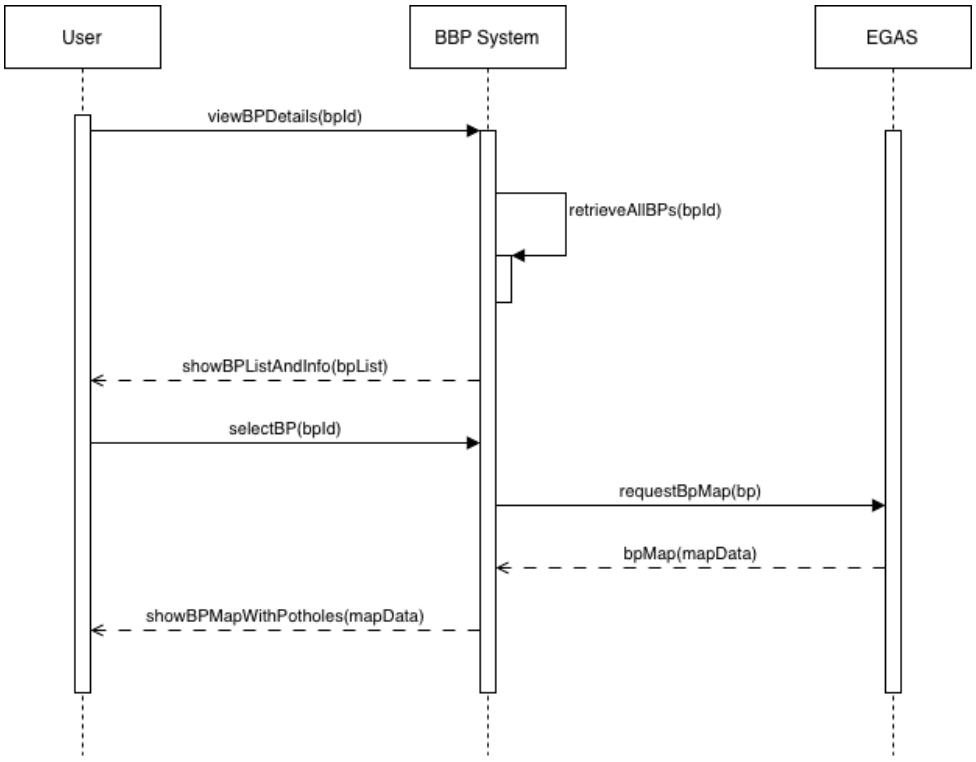


Figura 3.16: Use Case 13.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.2.4 Traceability Matrix

Rx \ UCy	UC1	UC2	UC3	UC4	UC5	UC6	UC7	UC8	UC9	UC10	UC11	UC12	UC13
R1	✓												
R2		✓											
R3			✓										
R4				✓									
R5					✓								
R6					✓								
R7					✓								
R8					✓								
R9					✓								
R10						✓							
R11							✓						
R12							✓						
R13							✓						
R14								✓					
R15								✓					
R16								✓					
R17								✓					
R18					✓			✓					
R19					✓			✓					
R20							✓						
R21										✓			
R22											✓		
R23											✓	✓	
R24											✓	✓	
R25											✓	✓	
R26													✓
R27									✓				

Table 14: Requirements/Use Cases Mapping Table

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.2.5 Goals, Requirements and Domain Assumptions Mapping

Goal	Description	Related Requirements	Related Domain Assumptions
G1	RUs should be able to store information about their trips, like distance covered, average speed, and, if available, also meteorological information.	R1, R2, R3, R5, R6, R7, R8, R9, R10, R18, R19	DA6, DA7, DA8
G2	RUs should be able to manually insert in BBP information about BPs, like status or obstacles.	R1, R2, R3, R11, R12, R13, R20	DA2, DA3, DA9
G3	RUs should be able to insert in BBP information about BPs, confirming or correcting what has been automatically retrieved by the BBP system.	R1, R2, R3, R14, R15, R16, R17, R18, R19	DA4, DA5, DA6, DA7, DA9
G4	RUs should be able to publish the BPs they previously inserted.	R1, R2, R3, R11, R14, R21, R27	DA2, DA3, DA4, DA5, DA6, DA7
G5	Any kind of user, even non-registered ones, by entering in BBP an origin and a destination, can visualize all the BPs between these two points previously published by other RUs ordered based on their score.	R1, R2, R3, R4, R21, R22, R23, R24, R25, R26	DA1, DA2, DA3, DA4, DA5, DA6, DA7, DA9

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.3 Performance Requirements

This section details the non-functional performance requirements that BBP must satisfy to ensure an efficient, responsive and scalable service across all the supported devices.

3.3.1 Latency Requirements

BBP must guarantee a smooth and responsive user experience during the navigation and the data acquisition. To ensure a great Quality of Experience, the system shall ensure that user interactions exhibit a response time $\leq$ **300 ms** under normal operating conditions.
Map rendering and BP visualization shall complete **within 700 ms** after the relevant data is retrieved.
Real-time features such as GPS updates, speed computation and automatic BP inference must process new data with update cycles executed at least once per second, with sensor-based inference delay **no greater than 500 ms** from acquisition.

3.3.2 Storage Requirements

BBP stores data elements such as GPS coordinates, trip statistics, BP street lists, pothole reports, and related metadata. To remain efficient, the system shall require on average **20–40 MB per active RU**, applying compression to dense GPS and sensor data whenever possible. All trip, BP, and EBP data shall be persistently stored with appropriate backup and recovery mechanisms to prevent data loss.

3.3.3 Scalability Requirements

The BBP system,in its first release, must support **up to 100,000 concurrent users** browsing EBPs in guest mode and **up to 50,000 concurrent RUs** performing operations such as trip recording or automated BP insertion. These numbers are the product of an initial estimation of the number of people that could be interested in the functionalities offered by BBP. The architecture shall scale elastically to handle temporary traffic peaks **up to twice the normal load**. If the average traffic increases over time, horizontal scaling can be performed to manage more users.

3.3.4 Throughput Requirements

The BBP system shall sustain **at least 1000 EBP search requests per second**, while publishing new BPs and updating EBPs must remain responsive even with **tens of thousands of simultaneous contributors**, without noticeable performance degradation.

3.4 Design Constraints

3.4.1 Standards compliance

The BBP system shall be composed of a **mobile client application** and a **backend server**. The mobile client shall be implemented using mobile OS SDKs (or a cross-platform mobile framework) and shall communicate with the BBP backend through HTTPS. The backend shall expose a RESTful API for the mobile client. Interactions with external services (XMS, EGAS) shall be publicly documented APIs. Sensor access on the client shall rely on mobile operating system APIs (e.g., location services/GPS, accelerometer, gyroscope) and their runtime permission mechanisms.

3.4.2 Hardware limitations

Automatic BP acquisition is constrained by the user's mobile device: BBP depends on the **availability and accuracy of GPS and sensors**. Trip recording and sensor polling must be lightweight enough to avoid excessive battery consumption and remain functional on mid-tier smartphones.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.4.3 Other Constraints

BBP depends on external services: XMS for weather, EGAS for path validation, whose availability and accuracy cannot be controlled by the system. Biking activity detection is constrained by speed-based thresholds defined by the domain. Finally, BBP requires a stable internet connection and a supported mobile operating system version with access to the required sensors and granted permissions, since synchronization with the backend and external API calls require connectivity.

3.5 Software System Attributes

3.5.1 Reliability

BBP must reliably acquire sensor data and handle temporary failures of external services by degrading gracefully rather than interrupting the user workflow. Data recorded during trip or BP insertion must be stored atomically to avoid partial or corrupted entries.

3.5.2 Availability

As a **client-server system**, BBP should remain available under normal network conditions. The backend services should be reachable for authentication, data synchronization, EBP search and publication. When external services are unreachable, BBP must still operate, omitting weather data or validation/map results when needed.

3.5.3 Security

All communication must occur over **HTTPS** to protect sensor data, location traces, and user credentials. Authentication must prevent unauthorized access, and user-generated data must be isolated so that no RU can manipulate another user's content. Interactions with external services must not expose sensitive information.

3.5.4 Maintainability

BBP should be modular, with clearly separated components for trip recording, BP acquisition, BP validation, and external service integration. This separation facilitates updates to individual modules without impacting the rest of the system.

3.5.5 Portability

The BBP mobile client should run on **major mobile platforms** (e.g., Android and iOS) and adapt to different smartphone screen sizes. The client shall rely on standard mobile OS APIs for sensor access and permissions.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

4 Formal Analysis Using Alloy

In this section, a formal analysis of a part of the system using the formal language Alloy is presented. The analysis is focused on the evolution of a BP's state through the phase of insertion in BBP until its publication as represented in figure 2.2. The analysis focuses on this aspect rather than others because BPs are the only entities of the system whose state representation is non-trivial and, therefore, a formal model could be helpful to check that some BPs' properties related to their state are verified if certain conditions are met.

4.1 Alloy Model

As mentioned above, the model describes BPs and the evolution of their state from creation to publication as shown in figure 2.2. In order to do so, eight *sig* have been defined:

- **Status**: an *enum* with all the possible statuses a BP can have
- **Bool**: an *enum* to express Boolean values
- **BikePath**: an *abstract sig* representing the concept of BP. Its two concrete *sig* represent the two different ways that can be used to insert a BP in BBP
- **ManuallyInsertedBikePath**: *concrete sig* of BikePath that represents BPs inserted using manual mode
- **AutomaticallyInsertedBikePath**: *concrete sig* of BikePath that represents BPs inserted using automated mode
- **StreetNode**: a *sig* representing the node of a list of streets associated with a single BP. It is defined as a *var sig* because users can modify the list of streets in manual mode if it is not validated by the EGAS, so the idea is that the list can change over time. However, the field nextStreet is immutable over time, so a change of the street list corresponds to a change of the origin of the list (var field in BikePath) and the creation of a new list
- **Street**: a *sig* representing the street. Unlike the previous *sig*, this one is not related to a BP, it just represents a street in the world and, therefore, it is immutable over time
- **PotholeNode**: a *sig* similar to StreetNode but for the concept of pothole. It behaves in the same way as StreetNode

Note: all the fiels of the *var sig* are defined as *var*, however some of them are immutable over time. The *facts* **streetNodeImmutability** and **potholeNodeImmutability** impose this constraint. These fields were not declared *static* from the beginning, because the Alloy tool issues warnings when compiling the code in this case.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Each state of 2.2 is represented through particular values that the Bool fields of the BikePath sig have:

- **Newly Created**: all fields are empty and all Bool fields are false
- **Streets Inserted** and **Waiting for EGAS's validation**: infoInserted = True and validatedByEGAS = published = False
- **Waiting for User's validation**: infoInserted = True and correctedByUser = confirmedByUser = False
- **Inserted in BBP for BPs inserted using manual mode**: infoInserted = validatedByEGAS = True and published = False
- **Inserted in BBP for BPs inserted using automated mode**: infoInserted = correctedByUser = True and published = confirmedByUser = False if the user corrected some false positives or infoInserted = confirmedByUser = True and published = correctedByUser = False if the user confirmed what the system has detected
- **Published in BBP**: published = True

Here follows the alloy model with all the facts that impose the constraints needed for a faithful representation of the BPs:

```
enum Status {Optimal, Medium, Sufficient, RequiresMaintenance}

enum Bool {True, False}

var abstract sig BikePath {
        var status: lone Status,
        var origin: lone StreetNode,
        var infoInserted:  one Bool,
        var published: one Bool
}

var sig ManuallyInsertedBikePath extends BikePath {
        var validatedByEGAS: one Bool
}

var sig AutomaticallyInsertedBikePath extends BikePath {
        var correctedByUser: one Bool,
        var confirmedByUser: one Bool
}

var sig StreetNode {
        var bikePath: one BikePath,
        var street: one Street,
        var potholesDetected: lone PotholeNode,
        var end: one Bool,
        var nextStreet: lone StreetNode
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

```
}

sig Street {
        adjacentStreets: some Street,
        length: one Int
} {length > 0}

var sig PotholeNode {
        var isLast: one Bool,
        var streetNode: one StreetNode,
        var distanceFromOrigin: one Int,
        var nextPothole: lone PotholeNode
} {distanceFromOrigin > 0}

/* in the initial instant the world is empty except for the streets that are always
the same throughout all the states */
fact initialState {
        no BikePath and no StreetNode and no PotholeNode
}

/********************** Creation and Insertion Constraints **********************/

/* this predicate is true when a BP has been created in the current instant */
pred newlyCreatedBikePath[bp: BikePath] {
        (before bp not in BikePath) and bp in BikePath
}

/* the only variable fields of StreetNode should be potholesDetected */
fact streetNodeImmutability {
        always (all sn: StreetNode | ((after sn in StreetNode) implies (sn.bikePath =
        (sn.bikePath)' and sn.street = (sn.street)' and sn.end = (sn.end)' and
        sn.nextStreet = (sn.nextStreet)')))
}

/* the fields of PotholeNode are immutable */
fact potholeNodeImmutability {
        always (all pn: PotholeNode | ((after pn in PotholeNode) implies (pn.isLast =
        (pn.isLast)' and pn.streetNode = (pn.streetNode)' and pn.distanceFromOrigin =
        (pn.distanceFromOrigin)'  and pn.nextPothole = (pn.nextPothole)')))
}

/* regarding the two facts above, I did not directly declare the fields static because
of warnings given by the tool */

/* once created the bike path cannot disappear from the world */
fact bikePathPermanence {
        always (all bp: BikePath |  (bp in ManuallyInsertedBikePath implies after bp
        in ManuallyInsertedBikePath) and
        (bp in AutomaticallyInsertedBikePath implies after bp in
        AutomaticallyInsertedBikePath)
        )
}

/* if a BP has just been created it has no information inserted in that instant */
fact newlyCreatedBikePathConstraints {
        always (
                all bp: BikePath | (newlyCreatedBikePath[bp] implies (
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

```
                        no bp.origin and no bp.status and bp.infoInserted = False and
                        bp.published = False and
                        ((bp in ManuallyInsertedBikePath) implies bp.validatedByEGAS =
                        False) and
                        ((bp in AutomaticallyInsertedBikePath) implies
                        (bp.correctedByUser = False and bp.confirmedByUser = False))
                ))
        )
}

/* the predicate is true if the user has inserted info about the BikePath bp in manual
mode and the EGAS has not approved it yet */
pred infoInsertedManually[bp: BikePath] {
        bp in ManuallyInsertedBikePath and bp.infoInserted = True and
        bp.validatedByEGAS = False and bp.published = False
}

/* the predicate is true if the user has inserted info about the BikePath bp in
automated mode and has not approved or nor corrected the it yet */
pred infoInsertedAutomatically[bp: BikePath] {
        bp in AutomaticallyInsertedBikePath and bp.infoInserted = True and
        bp.correctedByUser = False and bp.confirmedByUser = False and bp.published =
        False
}

/* if a BP has just been created, then its only possible next state can be only one of
the two mentioned above */
fact infoInsertionStateConstraint {
        always ( all bp: BikePath | ( newlyCreatedBikePath[bp] implies
        after(infoInsertedManually[bp] or infoInsertedAutomatically[bp])))
}

pred infoInserted[bp: BikePath] {
         infoInsertedManually[bp] or  infoInsertedAutomatically[bp]
}

/* if a user has inserted information about the BP, then all the fields must have some
kind of information (either correct or wrong) and the field infoInserted will always
be True from then on */
fact informationInsertedConstraint {
        always ( all bp: BikePath | ( infoInserted[bp] implies (
                #bp.status = 1 and #bp.origin = 1 and always bp.infoInserted = True
        )))
}

/************************* Street Constraints ************************/

/* a StreetNode exists only if it is part of a list */
fact streetNodeExistanceConstraint {
        always (all sn: StreetNode | (sn in sn.bikePath.origin.*nextStreet))
}

/* if a street is the end of the path, then there is no next street. Moreover, each
path should always end */
fact pathEndingConstraint {
        always ( (all sn: StreetNode | (sn.end = True iff #sn.nextStreet = 0)) and
        (all bp: BikePath | bp.infoInserted = True implies (some sn: StreetNode |
        sn.bikePath = bp and sn.end = True)))
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

```
}

/* each node of a list of street nodes should refer to the same BikePath and is
already correct (adjacent streets and no repeated streets) if inserted through
automated mode (because we assume that the GPS retrieves that kind of information
correctly, the false positive can only be generated by sensors) */
fact pathConstraint {
        always ((all bp: BikePath | #bp.origin = 1 implies bp.origin.bikePath = bp)
        and (all sn: StreetNode | (sn.end = False implies sn.bikePath =
        sn.nextStreet.bikePath) and ((sn.bikePath in AutomaticallyInsertedBikePath and
        sn.end = False) implies (sn.nextStreet.street in sn.street.adjacentStreets and
        sn.street not in sn.^nextStreet.street))
        ))
}

/* a street cannot be adjacent to itself and if s1 is adjacent to s2, then s2 is
adjacent to s1 */
fact adjacencyConstraint {
        (all s: Street | s not in s.adjacentStreets) and
        (all s1, s2: Street | (s1 != s2 implies (s1 in s2.adjacentStreets iff s2 in
        s1.adjacentStreets)))
}

/************************* Potholes Constraints *************************/

/* a PotholeNode exists only if it is part of a list */
fact potholeNodeExistanceConstraint {
        always (all pn: PotholeNode | (pn in
        pn.streetNode.potholesDetected.*nextPothole))
}

/* given a StreetNode sn, returns the set of all streets from sn1 up to sn2 (excluded)
*/
fun streetsUpTo[sn1, sn2: StreetNode]: set Street {
    (sn1.*nextStreet - sn2.*nextStreet).street
}

/* given a StreetNode sn, returns the sum of the lentgh of all the streets before sn
in the BP */
fun distanceFromOrigin(sn: StreetNode) : Int {
        sum streetsUpTo[sn.bikePath.origin, sn].length
}

/* all potholes of a same StreetNode sn must refer to sn, must have a coherent
distance from the origin and if it is the last on the list, it must not have a
successor */
fact potholeConstraints {
        always ( (all sn: StreetNode |  #sn.potholesDetected = 1 implies
        sn.potholesDetected.streetNode = sn) and (all pn: PotholeNode | (
                (pn.isLast = True iff #pn.nextPothole = 0) and
                (pn.isLast = False implies pn.nextPothole.streetNode = pn.streetNode)
                and
                (pn.distanceFromOrigin >= distanceFromOrigin[pn.streetNode] and
                pn.distanceFromOrigin < distanceFromOrigin[pn.streetNode] +
                pn.streetNode.street.length) and
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

```
                (pn.isLast = False implies  pn.nextPothole.distanceFromOrigin >
                pn.distanceFromOrigin)                                  // the
                potholes in the list are ordered by increasing distance from the
                origin
        )))
}

/************************ Manual Mode Constraints ************************/

/* returns true only if the EGAS validates the streets composing the BP */
pred validatedByEGAS[bp: BikePath] {
        bp in ManuallyInsertedBikePath and bp.infoInserted = True and
        bp.validatedByEGAS = True and bp.published = False
}

/* returns true if a path can be validated by the EGAS. A path can be validated by the
EGAS if it is formed by adjacent streets and has no repeated streets in it. */
pred validableByEGAS[bp: BikePath] {
        bp in ManuallyInsertedBikePath and bp.validatedByEGAS = False and
        bp.infoInserted = True and bp.published = False and all sn: StreetNode |
        (sn.bikePath = bp implies (
                (sn.end = False implies sn.nextStreet.street in
                sn.street.adjacentStreets) and (sn.end = False implies sn.street not
                in sn.^nextStreet.street)
        ))
}

/* since the only variable field of a streetNode is potholesDetected, a list of
StreetNode is unchanged in the next instant if all the fields potholeDetected are
unchanged and the BikePath has the same origin (even the nextStreet field is static
so, a different list of street is just a list with a different origin or different
potholes) */
pred streetListUnchanged[bp: BikePath] {
        (all sn: StreetNode | sn.bikePath = bp implies sn.potholesDetected =
        (sn.potholesDetected)') and bp.origin = (bp.origin)'
}

/* if the info manually inserted is correct then, in the next instant, it will be
validated by the EGAS, so it cannot be modified anymore */
fact EGASValidationConstraint {
        always (all bp: BikePath |  validableByEGAS[bp] implies ((after
        validatedByEGAS[bp]) and bp.status = (bp.status)' and
        streetListUnchanged[bp]))
}

/* if a user is modifying a wrong path before the EGAS validation, then the
unmodifiable fields should be the same over all instants */
fact manualModificationConstraint {
        always (all bp: BikePath | infoInsertedManually[bp] implies (bp.published =
        (bp.published)' and bp.infoInserted = (bp.infoInserted)' and (not
        validableByEGAS[bp] implies after bp.validatedByEGAS = False)))
}

/************************ Automated Mode Constraints ************************/

/* if a BP is inserted using automated mode user can either correct or confirm the
data retrieved, not both. This happens in the next instant and is permanent */
fact userValidationConstraint {
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

```
        always (all bp: BikePath | ( infoInsertedAutomatically[bp] implies (
                after always ((bp.correctedByUser = True and bp.confirmedByUser =
                False) or (bp.correctedByUser = False and bp.confirmedByUser = True))
        )))
}

/* returns true if the BP is unchanged in the next instant (including the potholes
detected, excluding the info regarding the two different concrete sig) */
pred bikePathUnchanged[bp: BikePath] {
        bp.origin = (bp.origin)' and bp.origin.*nextStreet.potholesDetected =
        (bp.origin.*nextStreet.potholesDetected)' and
        bp.status = (bp.status)' and bp.infoInserted = (bp.infoInserted)' and
        bp.published = (bp.published)'
}

/* if a BP is confirmed then it is unchanged */
fact BPConfirmationConstraint {
        always  (all bp: BikePath | ((bp.confirmedByUser = False and
        (bp.confirmedByUser)' = True) implies bikePathUnchanged[bp]))
}

/* in automated mode users can only modify the potholes detected or the status,
because we assume that the gps retrieves data correctly (the street list is correct)
*/
fact BPCorrectionConstraint {
        always  (all bp: BikePath | ((bp.correctedByUser = False and
        (bp.correctedByUser)' = True) implies (
                bp.origin = (bp.origin)' and
                bp.infoInserted = (bp.infoInserted)'and
                bp.published = (bp.published)' and (
                        bp.status != (bp.status)' or
                        bp.origin.*nextStreet.potholesDetected !=
                        (bp.origin.*nextStreet.potholesDetected)')
        )))
}

/************************* Post-Insertion Constraints ************************/

/* returns true if the BP has been corrected by a user in automated mode */
pred correctedByUser[bp: BikePath] {
        bp in AutomaticallyInsertedBikePath and bp.infoInserted = True and
        bp.confirmedByUser = False and bp.correctedByUser = True and bp.published =
        False
}

/* returns true if the BP has been confirmed by a user in automated mode */
pred confirmedByUser[bp: BikePath] {
        bp in AutomaticallyInsertedBikePath and bp.infoInserted = True and
        bp.confirmedByUser = True and bp.correctedByUser = False and bp.published =
        False
}

/* returns true if the insertion is completed */
pred insertionCompleted[bp: BikePath] {
        validatedByEGAS[bp] or correctedByUser[bp] or confirmedByUser[bp]
}
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

```
/* once the insertion is completed, the info cannot be modified anymore (it can only
be published) */
fact insertionFinalizedConstraint {
        always (all bp: BikePath | (insertionCompleted[bp] implies (
                bp.origin = (bp.origin)' and bp.origin.*nextStreet.potholesDetected =
                (bp.origin.*nextStreet.potholesDetected)' and
                bp.status = (bp.status)' and bp.infoInserted = (bp.infoInserted)'and
                (bp in ManuallyInsertedBikePath implies bp.validatedByEGAS =
                (bp.validatedByEGAS)') and
                (bp in AutomaticallyInsertedBikePath implies (bp.confirmedByUser =
                (bp.confirmedByUser)' and bp.correctedByUser = (bp.correctedByUser)'))
        )))
}

/* once the BP is published it cannot be made private again (like the fact above but
now also the field "published" is immutable) */
fact publicationFinalizedConstraint {
        always (all bp: BikePath | (bp.published = True implies (bp.published =
        (bp.published)' and
                bp.origin = (bp.origin)' and bp.origin.*nextStreet.potholesDetected =
                (bp.origin.*nextStreet.potholesDetected)' and
                bp.status = (bp.status)' and bp.infoInserted = (bp.infoInserted)'and
                (bp in ManuallyInsertedBikePath implies bp.validatedByEGAS =
                (bp.validatedByEGAS)') and
                (bp in AutomaticallyInsertedBikePath implies (bp.confirmedByUser =
                (bp.confirmedByUser)' and bp.correctedByUser = (bp.correctedByUser)'))
        )))
}

/*NOTE: with these constraints a BP could either be eventually published or never
published*/
```

4.2 Alloy Results

To verify the soundness of the model defined above, predicates and assertions have been written and run using the Alloy tool. In this section, the results are displayed and their relevance is discussed.

4.2.1 Predicates

Three *predicates* have been run to show three different worlds in which BPs have been inserted and then published in three different ways:

- **pred showCreationToPublicationOfAnAutomaticallyInsertedBikePathWithoutCorrection**: world in which a BP is inserted using automated mode and the user confirms the info retrieved without correcting anything
- **pred showCreationToPublicationOfAnAutomaticallyInsertedBikePathWithCorrection**: world in which a BP is inserted using automated mode and the user corrects the information related to the potholes retrieved incorrectly by the sensors
- **pred showCreationToPublicationOfAManuallyInsertedBikePath**: world in which a BP is inserted using manual mode

```
enum Status {Optimal, Medium, Sufficient, RequiresMaintenance}
/********************** Predicates to show different worlds **********************/
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

```
pred showCreationToPublicationOfAnAutomaticallyInsertedBikePathWithoutCorrection{
        #BikePath = 0;
        #AutomaticallyInsertedBikePath = 1 and #BikePath = 1;
        #StreetNode = 3 and #PotholeNode = 1;
        all bp: BikePath | bp.confirmedByUser = True;
        all bp: BikePath | bp.published = False;
        all bp: BikePath | bp.published = True
}

run showCreationToPublicationOfAnAutomaticallyInsertedBikePathWithoutCorrection for 3
but 4 Street, 7 steps

pred showCreationToPublicationOfAnAutomaticallyInsertedBikePathWithCorrection{
        #BikePath = 0;
        #AutomaticallyInsertedBikePath = 1 and #BikePath = 1;
        #StreetNode = 2 and #PotholeNode = 1;
        all bp: BikePath | bp.correctedByUser = True  and #PotholeNode = 3;
        all bp: BikePath | bp.published = False;
        all bp: BikePath | bp.published = False;
        all bp: BikePath | bp.published = True
}

run showCreationToPublicationOfAnAutomaticallyInsertedBikePathWithCorrection for 3 but
4 Street, 7 steps

pred showCreationToPublicationOfAManuallyInsertedBikePath{
        #BikePath = 0 and #Street = 5;
        #ManuallyInsertedBikePath = 1 and #BikePath = 1;
        #StreetNode = 3 and #PotholeNode = 0 and all bp: BikePath | not
        validableByEGAS[bp];
        #StreetNode = 3 and all bp: BikePath | validableByEGAS[bp];
        all bp: BikePath | validatedByEGAS[bp];
        all bp: BikePath | bp.published = True
}

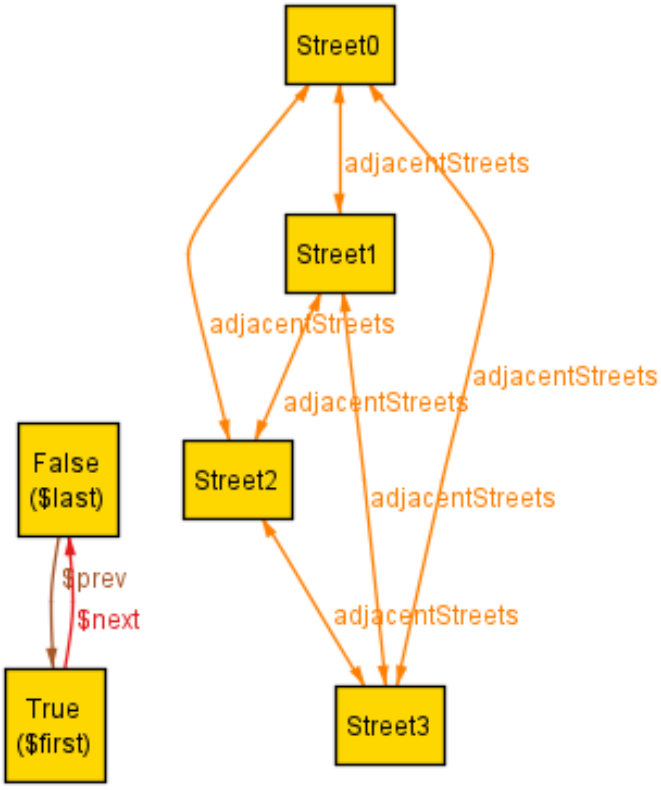
run showCreationToPublicationOfAManuallyInsertedBikePath for 3 but 6 Street, 4
StreetNode, 6 steps
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

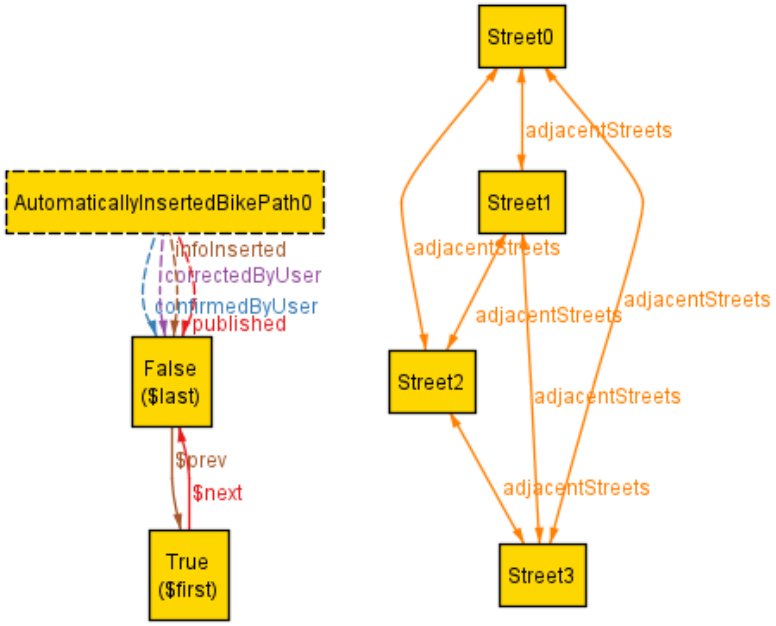
Here follows, for each *predicate*, a graphical representation of the world they generated (Status *enum* is omitted for readability):

pred showCreationToPublicationOfAnAutomaticallyInsertedBikePathWithoutCorrection:

- First instant: there is no BP but only the streets (constant over all instants)

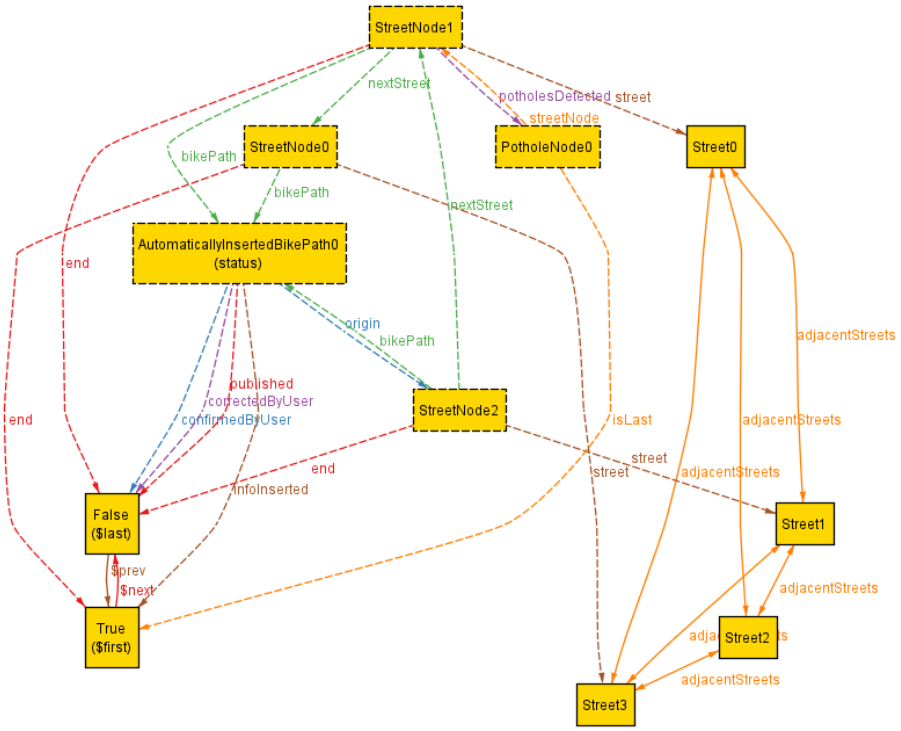


- Second instant: a BP has just been created using automated mode, so all its Bool fields are set to false

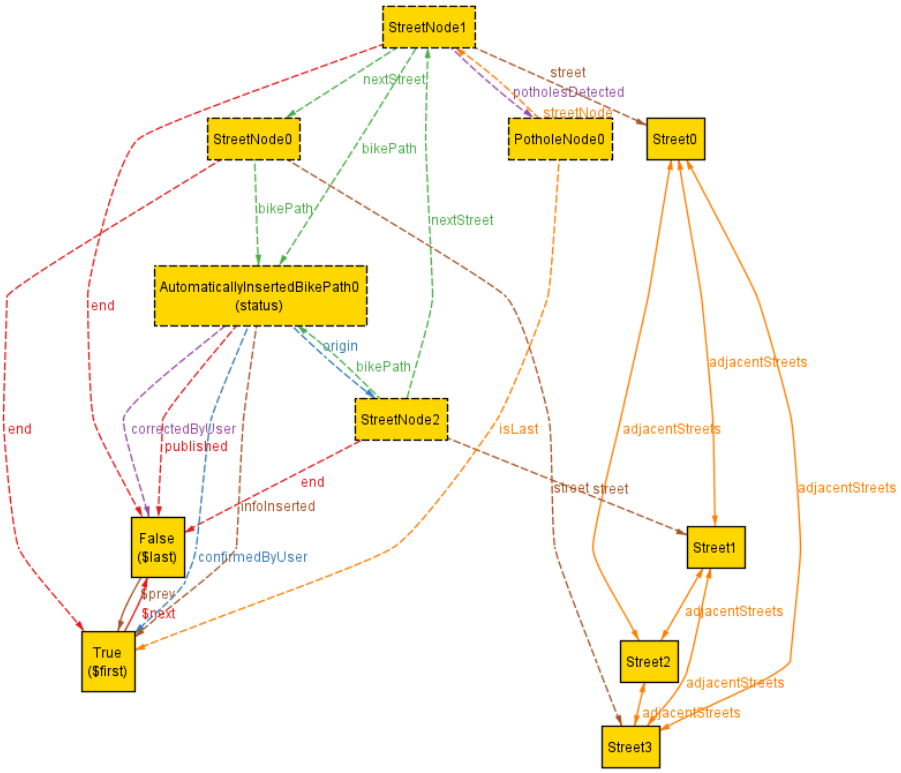


- Third instant: info related to the street has been entered automatically and the field infoInserted has been set to True

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

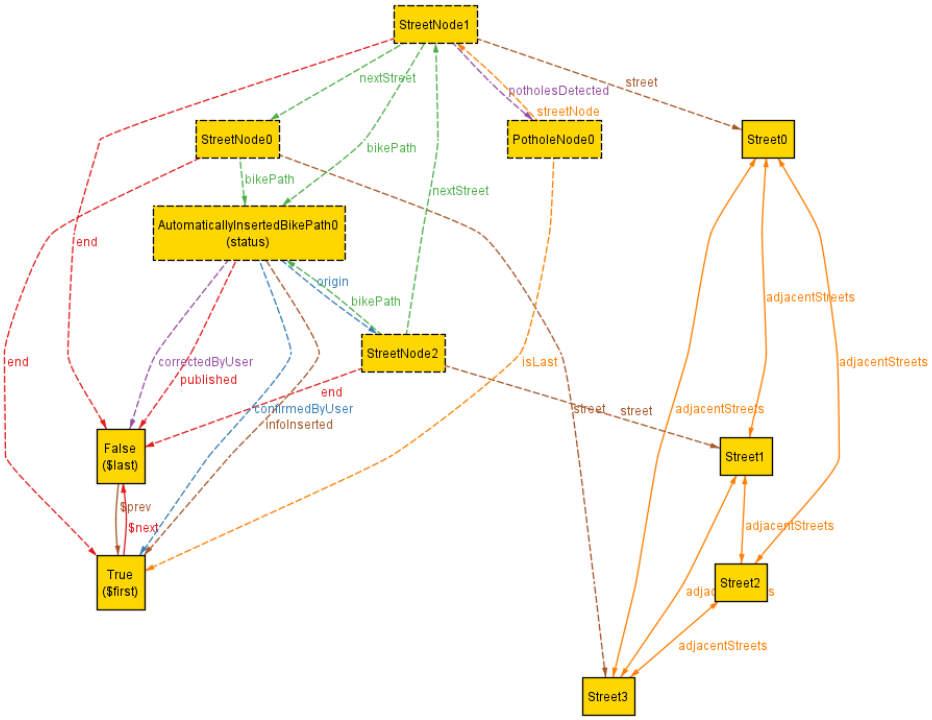


- Fourth instant: user confirmed the info retrieved, so everything is unchanged except for the field confirmedByUser which is set to True

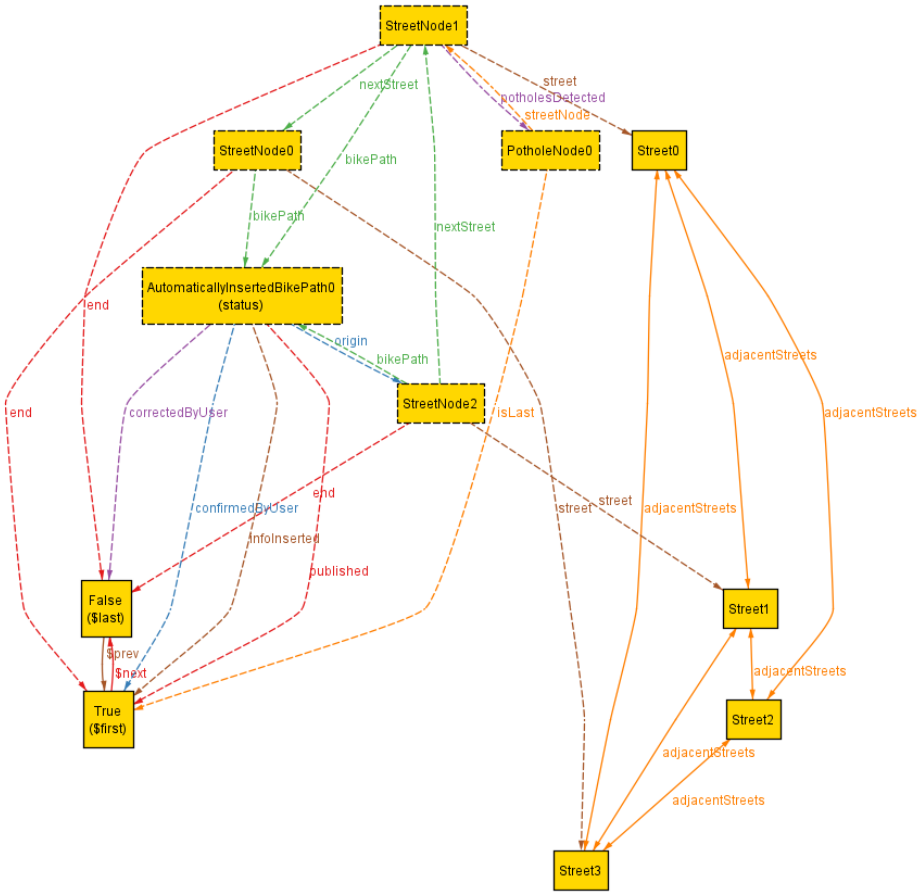


- Fifth instant: The world is unchanged because the user decided not to publish the BP

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved



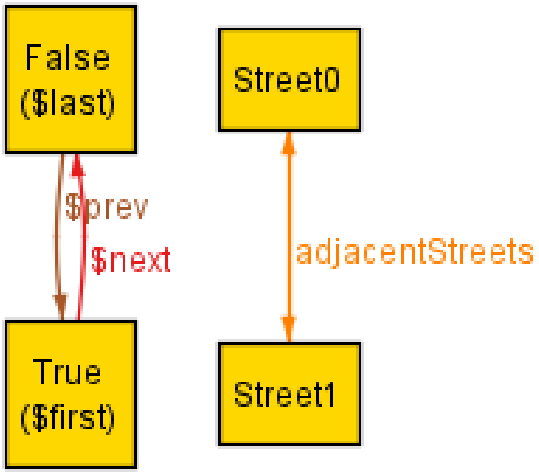
- Sixth instant: the user decided to publish their BP so everything is unchanged except for the published field which is set to True



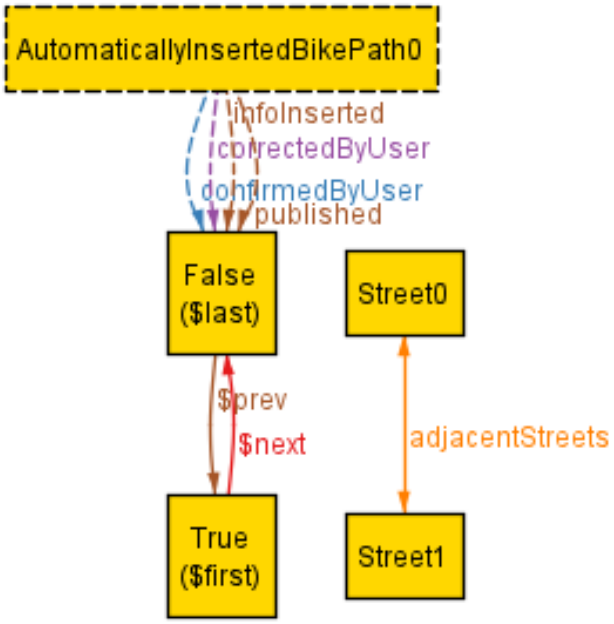
Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

pred showCreationToPublicationOfAnAutomaticallyInsertedBikePathWithCorrection:

- First instant: there is no BP, only the streets

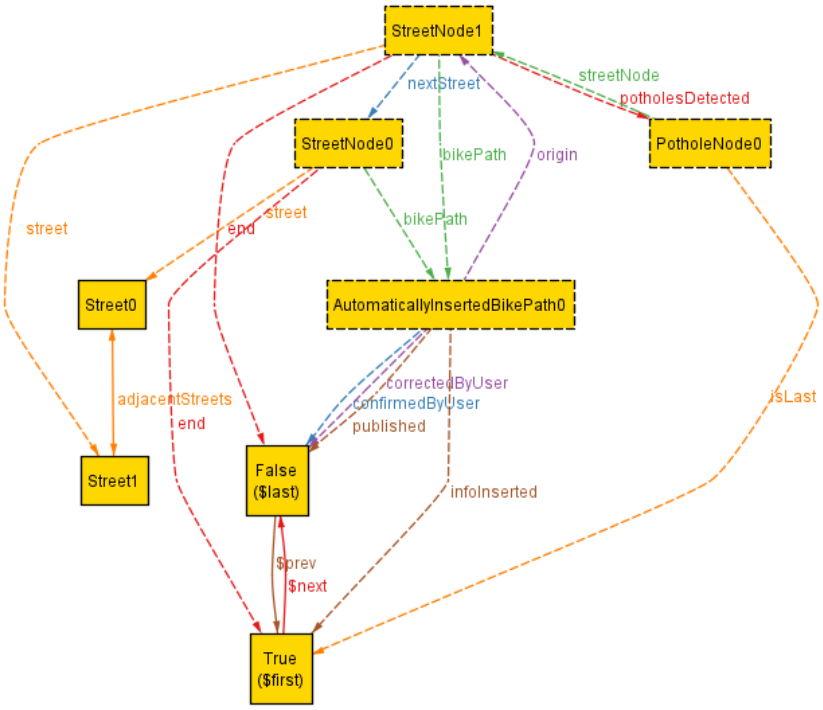


- Second instant: a BP has just been created using automated mode, so all its Bool fields are set to false



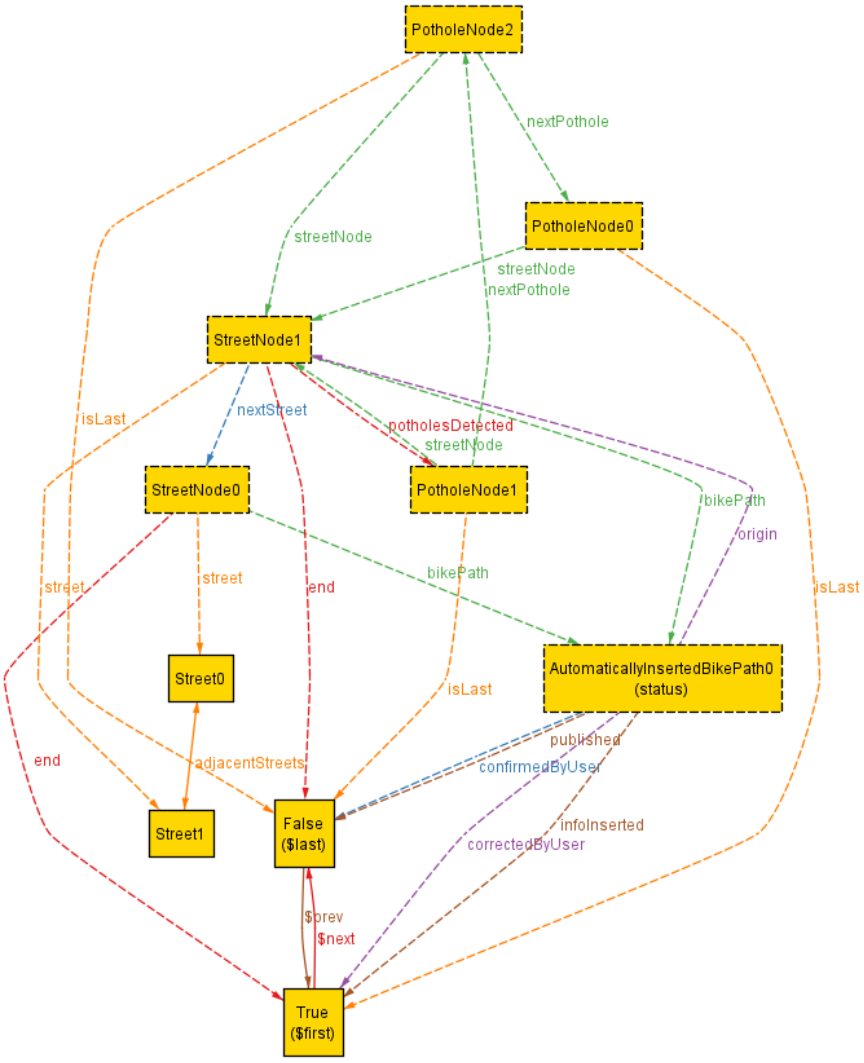
Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

- Third instant: info related to the street has been entered automatically and the field infoInserted has been set to True



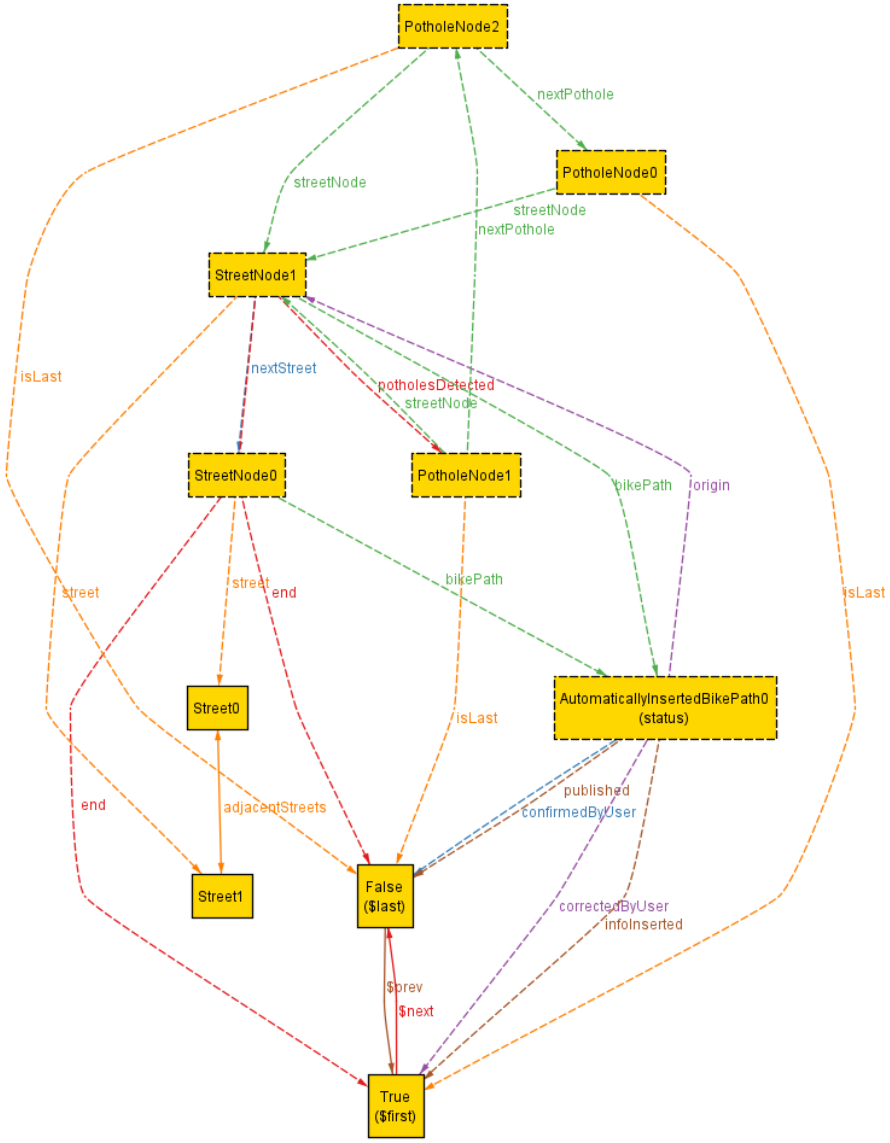
Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

- Fourth instant: the user has corrected the info about the potholes detected, in fact correctedByUser is set to True and the list of PotholeNode is different from before



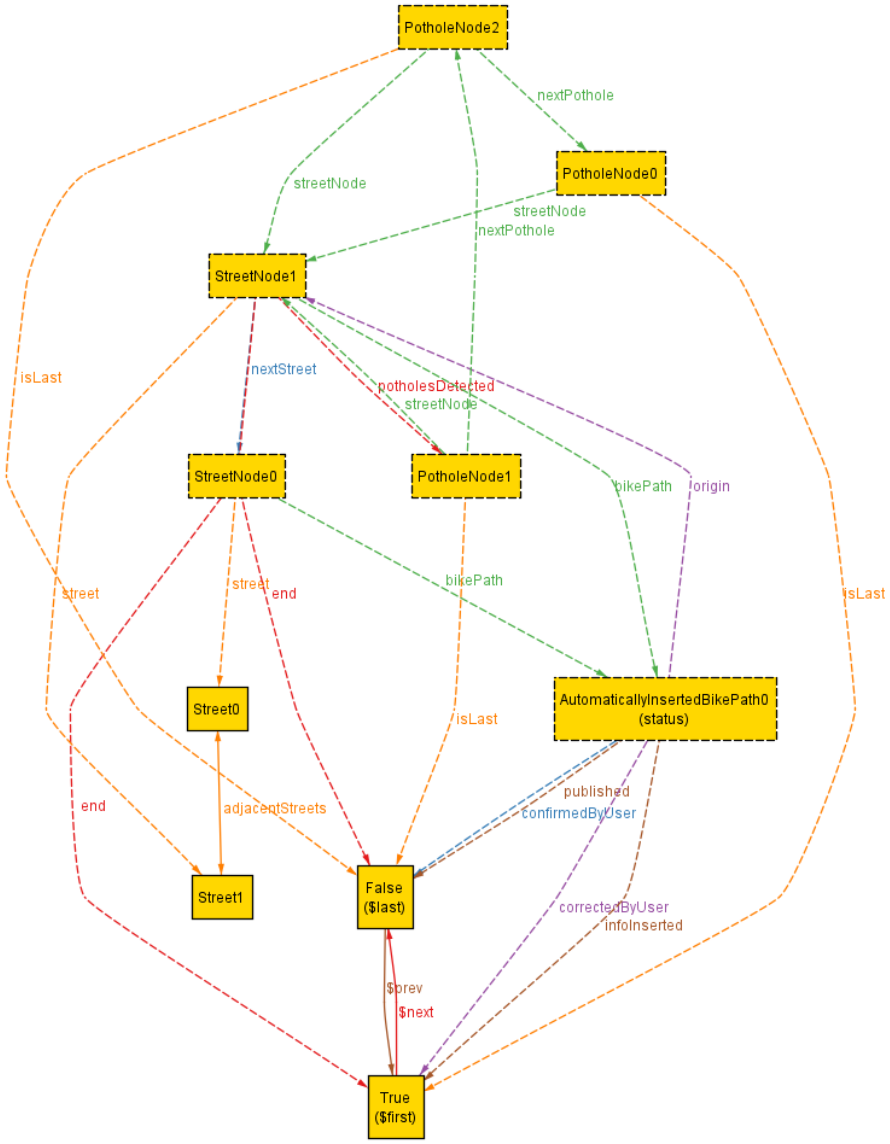
Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

- Fifth instant: the user has decided not to publish this BP so everything is unchanged



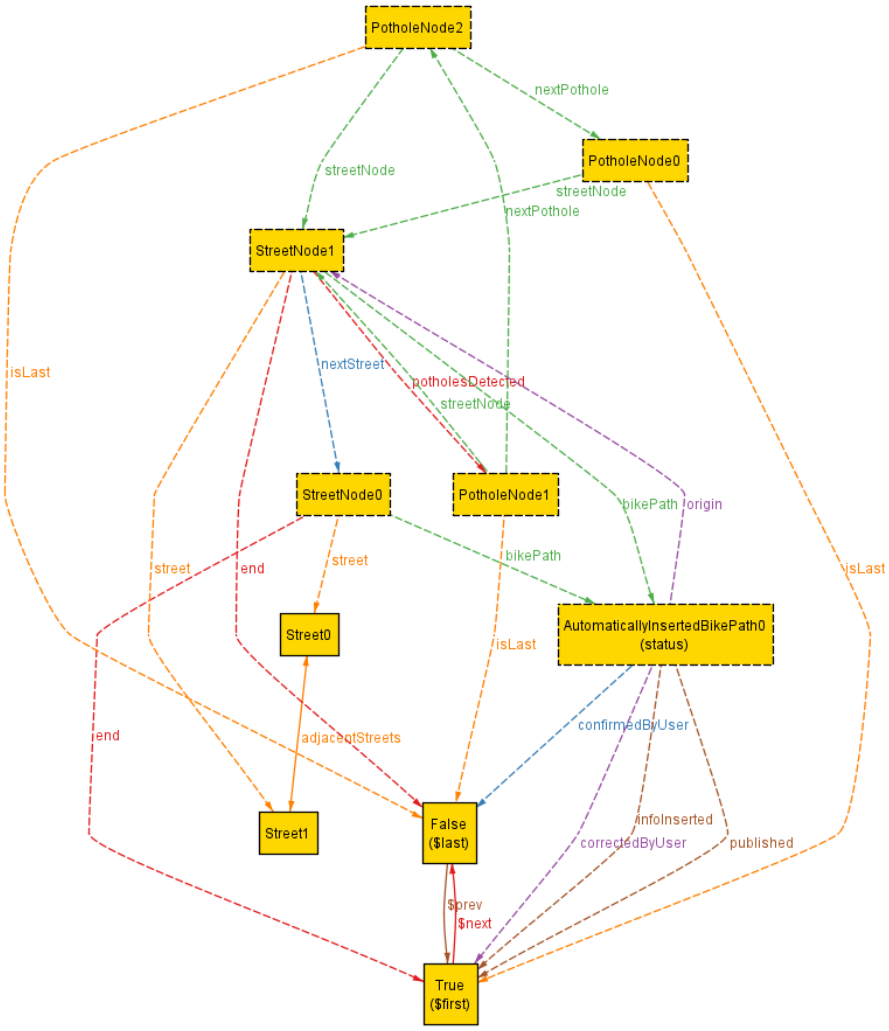
Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

- Sixth instant: same as instant five



Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

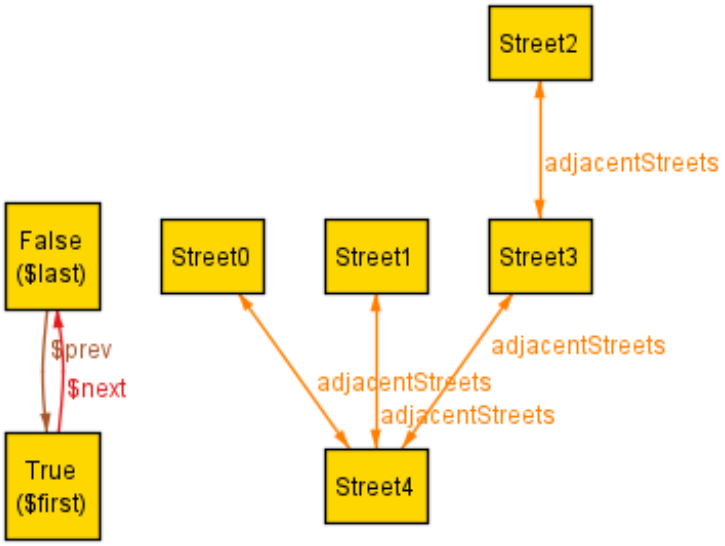
- Seventh instant: the user publishes the BP, so the field *published* is set to True



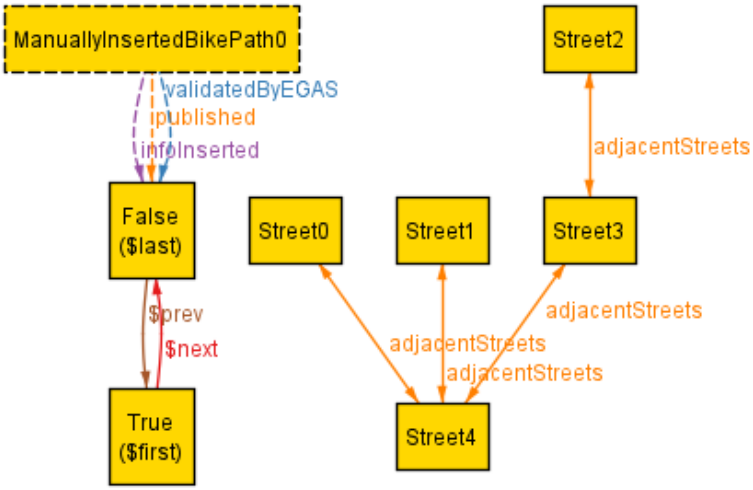
Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

pred showCreationToPublicationOfAManuallyInsertedBikePath:

- First instant: there is no BP, only the streets

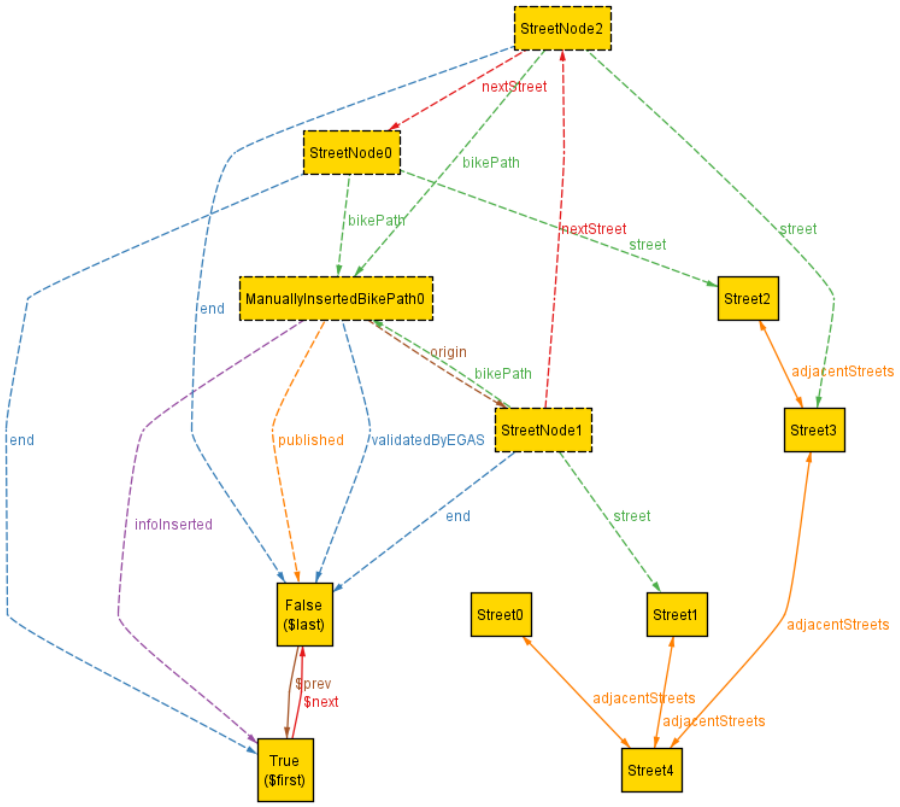


- Second instant: a BP has just been created using manual mode, so all its Bool fields are set to false

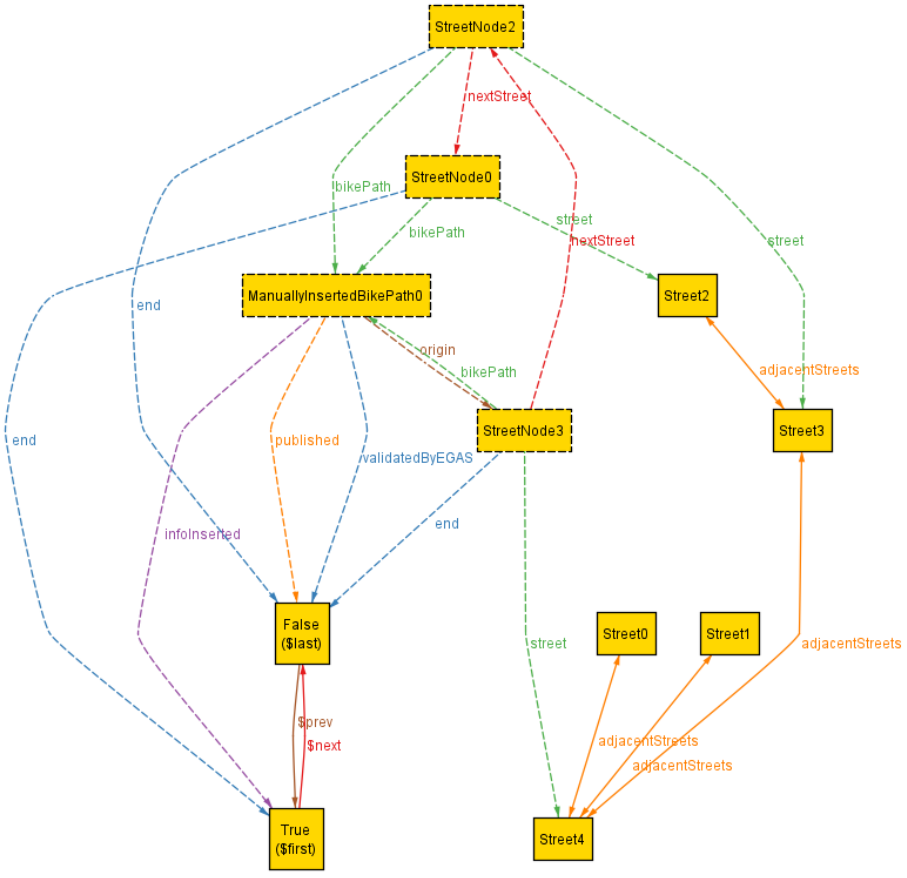


- Third instant: info related to the street has been manually entered by the user, so the field infoInserted has been set to True. However, the street list entered is not valid because StreetNode1 and StreetNode2 are associated with streets that are not adjacent. Moreover, StreetNode2 and StreetNode0 refer to the same street

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

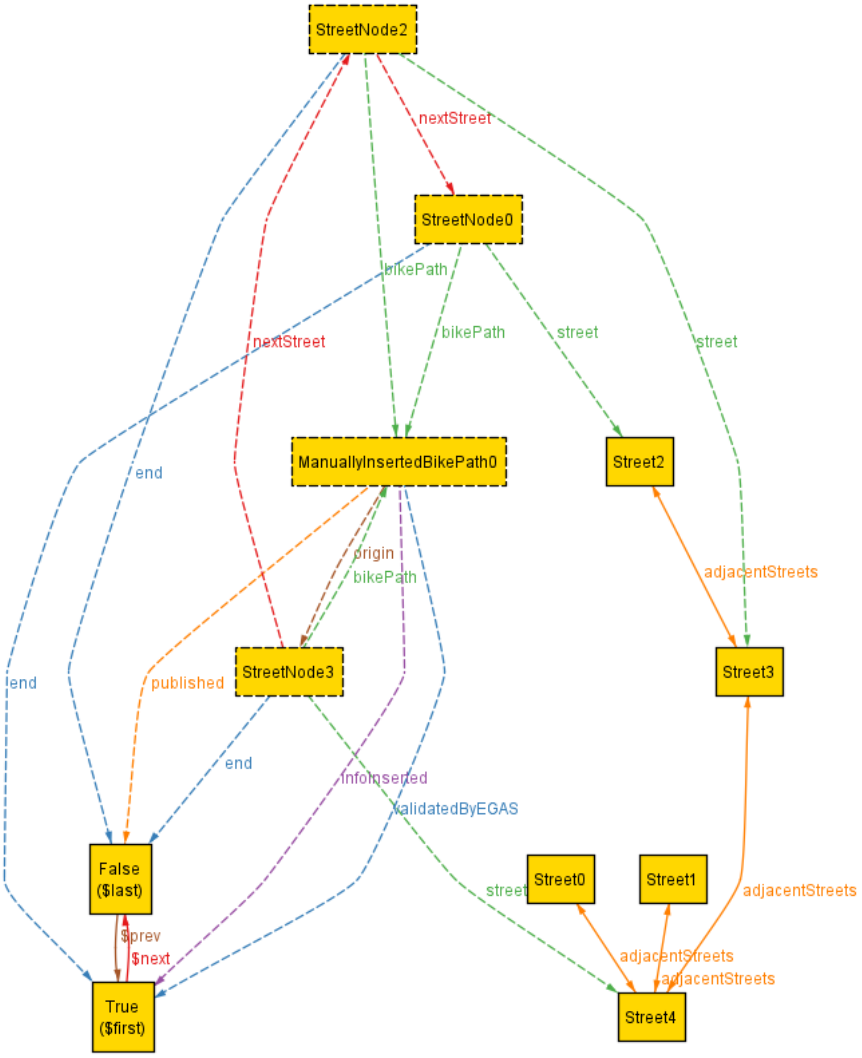


- Fourth instant: since in the previous instant the street list was not correct, the EGAS could not validate it, so the user modified it into a now proper path: all subsequent streets are adjacent and there are no repetitions



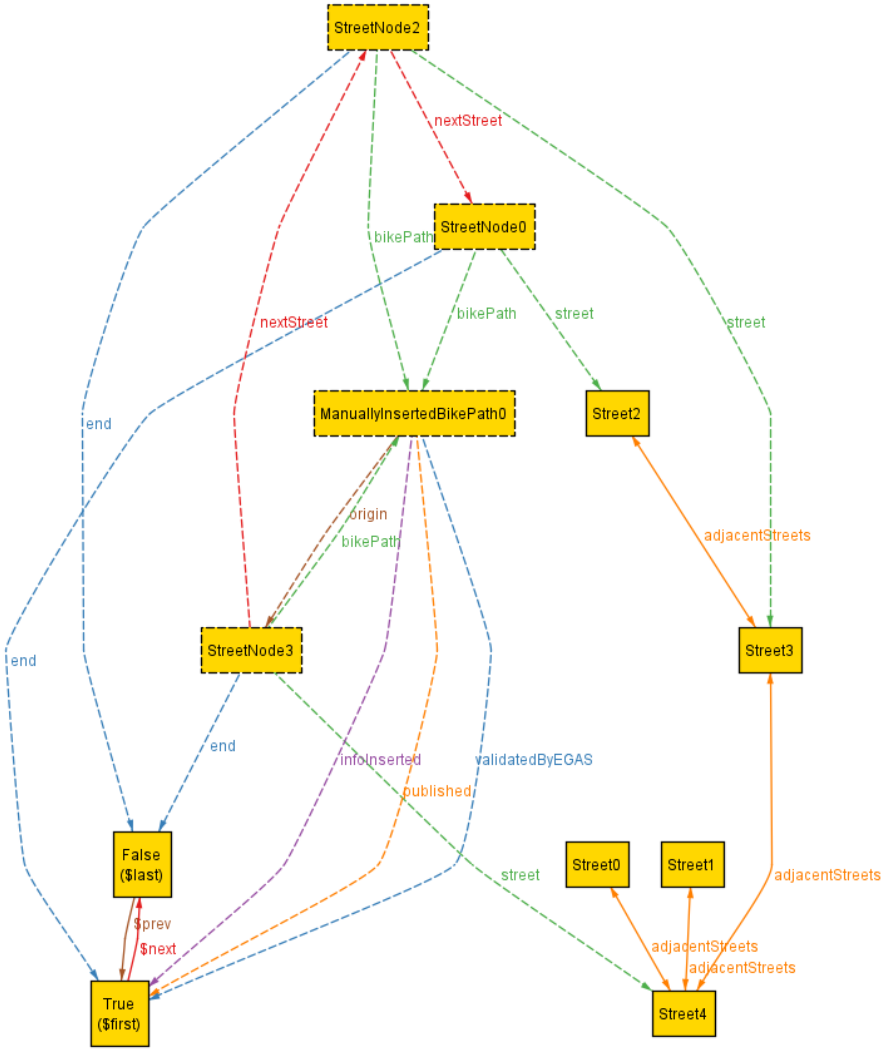
Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

- Fifth instant: the EGAS validated the BP, in fact the field validatedByEGAS is set to True, while everything else is unchanged



Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

- Sixth instant: the user publishes the BP so the field published is set to True



The worlds displayed above show how the model is coherent with the BPs' evolution of the states, as in 2.2. In particular, it is now clearer what a user can do in automated mode and cannot do in manual mode, for instance, in manual mode a user can modify any information about the BP before the validation by the EGAS, while in automated mode it is assumed that the GPS and the EGAS work correctly, so the only things the user can modify are the potholes detected by the sensors and not the list of the streets.

4.2.2 Assertions

Ten assertions have been checked, using the alloy tool, to show that some properties that are expected to be true hold in this model:

```
/************************ Assertions ************************/

/* checks that the publications can happen only if all information of a BP has been
correctly entered and validated */
assert bikePathsMustBeCompleteBeforePublished {
        always (all bp: BikePath | bp.published = True implies once
        insertionCompleted[bp])
}

check bikePathsMustBeCompleteBeforePublished for 5
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

```
/* checks if, when a path is inserted in automated mode and the users confirms it,
then the structure of the path never changes anymore */
assert bikePathImmutableAfterConfirmation {
        always (all bp: BikePath | (infoInsertedAutomatically[bp] and after
        confirmedByUser[bp]) implies streetListUnchanged[bp])
}

check bikePathImmutableAfterConfirmation for 5 but 1 BikePath

/* checks if a public path is correct in terms of "street list consistency" */
assert publicPathWithCorrectStreetList {
        always (all bp: BikePath | bp.published = True implies (all sn: StreetNode |
        (sn.bikePath = bp and sn.end = False) implies (sn.nextStreet.street in
        sn.street.adjacentStreets and sn.street not in sn.^nextStreet.street)))
}

check publicPathWithCorrectStreetList for 5 but 1 BikePath, 6 steps

/* checks the fact that the street list is always constant in time when the BP is
inserted through automated mode (the only thing varying are the potholes or the
status) */
assert usersCannotModifyStreetListInAutomaticallyInsertedBikePaths {
        always (all bp: AutomaticallyInsertedBikePath | infoInsertedAutomatically[bp]
        implies after always bp.origin = (bp.origin)')
}

check usersCannotModifyStreetListInAutomaticallyInsertedBikePaths for 5 but 1 BikePath

/* checks that if a user has inserted an invalid BP in manual mode and so the EGAS did
not confirm it, then the user must have modified it before validation by the EGAS */
assert userMustModifyPathNotConfirmedByEGAS {
        always (all bp: ManuallyInsertedBikePath | (infoInsertedManually[bp] and after
        (bp.validatedByEGAS = False) and eventually (bp.validatedByEGAS = True))
        implies
         eventually (bp.origin != (bp.origin)' or
         bp.origin.*nextStreet.potholesDetected !=
         (bp.origin.*nextStreet.potholesDetected)'))
}

check userMustModifyPathNotConfirmedByEGAS for 5 but 1 BikePath, 6 steps

/* checks that, when confirming an AutomaticallyInsertedBikePath, the path structure
and the status are unchanged in the future */
assert usersConfirmationShouldNotModifyBikePath {
        always (all bp: AutomaticallyInsertedBikePath | (bp.confirmedByUser = False
        and (bp.confirmedByUser)' = True) implies after always
        (streetListUnchanged[bp] and bp.status = (bp.status)'))
}

check usersConfirmationShouldNotModifyBikePath for 5 but 1 BikePath, 8 steps

/* checks that publication can happen only after at least three instants after the
creation of a BP (creation -> insertion -> validation (insertion finalized) ->
publication) */
assert publicationOnlyAfterThreeInstants {
        always (not (some bp: BikePath | bp.published = True and (before
        newlyCreatedBikePath[bp] or before before newlyCreatedBikePath[bp] )))
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

```
}

check publicationOnlyAfterThreeInstants for 5

/* checks that publication can never happen before the finalization of the insertion
*/
assert publicationOnlyAfterFinalizationOfInsertion {
        always (not (some bp: BikePath | bp.published = True and historically not
        insertionCompleted[bp]))
}

check publicationOnlyAfterFinalizationOfInsertion for 5

/* checks that after the insertion is finalized the BP will never go back in a state
in which is either not validated by the EGAS (manual mode) or has not been
confirmed/corrected by the user (automated mode) */
assert afterInsertionFinalizedCannotGoBackInInsertionState {
        always (all bp: BikePath | insertionCompleted[bp] implies after always not
        infoInserted[bp])
}

check afterInsertionFinalizedCannotGoBackInInsertionState for 5

/* checks that states related only to ManuallyInsertedBikePath are never connected to
the ones related only to AutomaticallyInsertedBikePath */
assert differentBikePathStatesCoherence {
        always ((not (some bp: ManuallyInsertedBikePath | insertionCompleted[bp] and
        after (correctedByUser[bp] or confirmedByUser[bp]))) and
                    (not (some bp: AutomaticallyInsertedBikePath |
                    insertionCompleted[bp] and after validatedByEGAS[bp]))
        )
}

check differentBikePathStatesCoherence for 5
```

All these *assertions* did not generate any counterexamples, so it is somewhat safe to assume that the ten properties reported above are true in this model (it is impossible to prove soundness because the tool can only perform bounded verification over a finite number of elements). Here the log of the tool for the assertions:

```
Executing "Check bikePathsMustBeCompleteBeforePublished for 5"
   Actual scopes: exactly 4 Status, exactly 1 Optimal, exactly 1 Medium, exactly 1
    Sufficient, exactly 1 RequiresMaintenance, exactly 2 Bool, exactly 1 True, exactly
     1 False, 5 BikePath, 5 ManuallyInsertedBikePath, 5 AutomaticallyInsertedBikePath,
     5 StreetNode, 5 Street, 5 PotholeNode, exactly 1 ordering/Ord, exactly 1 open$3/
    Ord
   Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
   1..10 steps. 1583603 vars. 37011 primary vars. 4314476 clauses. 7493ms.
   No counterexample found. Assertion may be valid. 1191ms.

Executing "Check bikePathImmutableAfterConfirmation for 5 but 1 BikePath"
   Actual scopes: exactly 4 Status, exactly 1 Optimal, exactly 1 Medium, exactly 1
    Sufficient, exactly 1 RequiresMaintenance, exactly 2 Bool, exactly 1 True, exactly
     1 False, 1 BikePath, 1 ManuallyInsertedBikePath, 1 AutomaticallyInsertedBikePath,
     5 StreetNode, 5 Street, 5 PotholeNode, exactly 1 ordering/Ord, exactly 1 open$3/
    Ord
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

```
   Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
   1..10 steps. 2335546 vars. 52786 primary vars. 6471151 clauses. 54389ms.
   No counterexample found. Assertion may be valid. 26099ms.

Executing "Check publicPathWithCorrectStreetList for 5 but 6 steps, 1 BikePath"
   Actual scopes: exactly 4 Status, exactly 1 Optimal, exactly 1 Medium, exactly 1
    Sufficient, exactly 1 RequiresMaintenance, exactly 2 Bool, exactly 1 True, exactly
     1 False, 1 BikePath, 1 ManuallyInsertedBikePath, 1 AutomaticallyInsertedBikePath,
     5 StreetNode, 5 Street, 5 PotholeNode, exactly 1 ordering/Ord, exactly 1 open$3/
    Ord
   Solver=sat4j Steps=1..6 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
   1..6 steps. 2627561 vars. 59077 primary vars. 7299456 clauses. 3468ms.
   No counterexample found. Assertion may be valid. 28300ms.

Executing "Check usersCannotModifyStreetListInAutomaticallyInsertedBikePaths for 5 but
     1 BikePath"
   Actual scopes: exactly 4 Status, exactly 1 Optimal, exactly 1 Medium, exactly 1
    Sufficient, exactly 1 RequiresMaintenance, exactly 2 Bool, exactly 1 True, exactly
     1 False, 1 BikePath, 1 ManuallyInsertedBikePath, 1 AutomaticallyInsertedBikePath,
     5 StreetNode, 5 Street, 5 PotholeNode, exactly 1 ordering/Ord, exactly 1 open$3/
    Ord
   Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
   1..10 steps. 3375281 vars. 74952 primary vars. 9454259 clauses. 4750ms.
   No counterexample found. Assertion may be valid. 1255ms.

Executing "Check userMustModifyPathNotConfirmedByEGAS for 5 but 6 steps, 1 BikePath"
   Actual scopes: exactly 4 Status, exactly 1 Optimal, exactly 1 Medium, exactly 1
    Sufficient, exactly 1 RequiresMaintenance, exactly 2 Bool, exactly 1 True, exactly
     1 False, 1 BikePath, 1 ManuallyInsertedBikePath, 1 AutomaticallyInsertedBikePath,
     5 StreetNode, 5 Street, 5 PotholeNode, exactly 1 ordering/Ord, exactly 1 open$3/
    Ord
   Solver=sat4j Steps=1..6 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
   1..6 steps. 3666306 vars. 81255 primary vars. 10286995 clauses. 4249ms.
   No counterexample found. Assertion may be valid. 21019ms.

Executing "Check usersConfirmationShouldNotModifyBikePath for 5 but 8 steps, 1
    BikePath"
   Actual scopes: exactly 4 Status, exactly 1 Optimal, exactly 1 Medium, exactly 1
    Sufficient, exactly 1 RequiresMaintenance, exactly 2 Bool, exactly 1 True, exactly
     1 False, 1 BikePath, 1 ManuallyInsertedBikePath, 1 AutomaticallyInsertedBikePath,
     5 StreetNode, 5 Street, 5 PotholeNode, exactly 1 ordering/Ord, exactly 1 open$3/
    Ord
   Solver=sat4j Steps=1..8 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
   1..8 steps. 4158165 vars. 91803 primary vars. 11699091 clauses. 19179ms.
   No counterexample found. Assertion may be valid. 23304ms.

Executing "Check publicationOnlyAfterThreeInstants for 5"
   Actual scopes: exactly 4 Status, exactly 1 Optimal, exactly 1 Medium, exactly 1
    Sufficient, exactly 1 RequiresMaintenance, exactly 2 Bool, exactly 1 True, exactly
     1 False, 5 BikePath, 5 ManuallyInsertedBikePath, 5 AutomaticallyInsertedBikePath,
     5 StreetNode, 5 Street, 5 PotholeNode, exactly 1 ordering/Ord, exactly 1 open$3/
    Ord
   Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
   1..10 steps. 5624703 vars. 113738 primary vars. 15669915 clauses. 19289ms.
   No counterexample found. Assertion may be valid. 3512ms.

Executing "Check publicationOnlyAfterFinalizationOfInsertion for 5"
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

```
   Actual scopes: exactly 4 Status, exactly 1 Optimal, exactly 1 Medium, exactly 1
    Sufficient, exactly 1 RequiresMaintenance, exactly 2 Bool, exactly 1 True, exactly
     1 False, 5 BikePath, 5 ManuallyInsertedBikePath, 5 AutomaticallyInsertedBikePath,
     5 StreetNode, 5 Street, 5 PotholeNode, exactly 1 ordering/Ord, exactly 1 open$3/
    Ord
   Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
   1..10 steps. 6599514 vars. 135273 primary vars. 18360101 clauses. 7038ms.
   No counterexample found. Assertion may be valid. 984ms.

Executing "Check afterInsertionFinalizedCannotGoBackInInsertionState for 5"
   Actual scopes: exactly 4 Status, exactly 1 Optimal, exactly 1 Medium, exactly 1
    Sufficient, exactly 1 RequiresMaintenance, exactly 2 Bool, exactly 1 True, exactly
     1 False, 5 BikePath, 5 ManuallyInsertedBikePath, 5 AutomaticallyInsertedBikePath,
     5 StreetNode, 5 Street, 5 PotholeNode, exactly 1 ordering/Ord, exactly 1 open$3/
    Ord
   Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
   1..10 steps. 7578013 vars. 156908 primary vars. 21066069 clauses. 10916ms.
   No counterexample found. Assertion may be valid. 4225ms.

Executing "Check differentBikePathStatesCoherence for 5"
   Actual scopes: exactly 4 Status, exactly 1 Optimal, exactly 1 Medium, exactly 1
    Sufficient, exactly 1 RequiresMaintenance, exactly 2 Bool, exactly 1 True, exactly
     1 False, 5 BikePath, 5 ManuallyInsertedBikePath, 5 AutomaticallyInsertedBikePath,
     5 StreetNode, 5 Street, 5 PotholeNode, exactly 1 ordering/Ord, exactly 1 open$3/
    Ord
   Solver=sat4j Steps=1..10 Bitwidth=4 MaxSeq=5 SkolemDepth=1 Symmetry=20 Mode=batch
   1..10 steps. 8555509 vars. 178393 primary vars. 23761011 clauses. 4733ms.
   No counterexample found. Assertion may be valid. 92ms.
```

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

5 Effort Spent

Here is a table showing the effort spent by the two members of the group:

Members	Chapter 1	Chapter 2	Chapter 3	Chapter 4
Costa Federico	5h	5h	20h	6h
De Simone Emanuele	5h	10h	6h	15h

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

6 References

6.1 References

- The Requirement Engineering and Design Project specification document A.Y. 2025-2026.
- Slides of the Software Engineering 2 course

6.2 Used Tools

- **draw.io** for UML diagrams
- **Overleaf** to write the document
- **GitHub** for versioning
- **Alloy** for formal analysis
- **ChatGPT/Gemini** used to learn LateX syntax and to correct some english sentences, here an example of a prompt and its output:

Input In recent years, cycling has become an increasingly popular and sustainable mean of transport.

However, cyclists often face difficulties to find accurate and updated information about BPs. Existing data are usually scattered, incomplete, or not maintained, making it hard to evaluate the quality, safety, and usability of available routes. This lack of trustable information affects both occasional and regular cyclists, who can encounter poorly maintained paths, obstacles, or unsuitable routes without notice.

Il testo sopra è corretto?

Output Il testo è molto chiaro e comprensibile, ma ci sono alcuni piccoli errori grammaticali e lessicali che lo rendono poco naturale ("unnatural") in inglese.

Ecco un'analisi dettagliata delle correzioni necessarie per renderlo perfetto.

Principali correzioni "Mean of transport" → "Means of transport"

In inglese, il sostantivo per "mezzo" (di trasporto) è sempre means, anche al singolare. "Mean" significa "cattivo" o "media matematica".

Corretto: "...a sustainable means of transport."

"Face difficulties to find" → "Face difficulties (in) finding"

L'espressione "difficulties to find" non è idiomatica. È meglio usare "face difficulties in finding", "have difficulty finding", oppure "struggle to find".

L'acronimo "BPs"

A meno che tu non abbia definito "BPs" (Bike Paths) in precedenza nel testo, non è un acronimo standard riconosciuto universalmente. È meglio scrivere "bike paths" o "cycling routes" per chiarezza.

"Trustable" → "Reliable"

"Trustable" esiste, ma è raramente usato in contesti formali o tecnici. Reliable (affidabile) o trustworthy (degno di fiducia) suonano molto più professionali.

"Data are" vs "Data is"

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Hai scritto "Existing data are". Questo è tecnicamente corretto (specialmente in ambito accademico scientifico), ma nell'inglese moderno "data" viene spesso trattato come singolare ("data is"). Puoi lasciarlo così se il contesto è molto formale, ma "data is" scorre meglio.

Versione Corretta (Consigliata) Ecco una versione che mantiene il tuo significato originale ma corregge gli errori e migliora la scorrevolezza:

"In recent years, cycling has become an increasingly popular and sustainable means of transport. However, cyclists often struggle to find accurate and updated information about bike paths. Existing data is usually scattered, incomplete, or not maintained, making it hard to evaluate the quality, safety, and usability of available routes. This lack of reliable information affects both occasional and regular cyclists, who may encounter poorly maintained paths, obstacles, or unsuitable routes without notice."

What we actually wrote: In recent years, cycling has become an increasingly popular and sustainable means of transportation. However, cyclists often face difficulties in finding accurate and up-to-date information about BPs. Existing data are usually scattered, incomplete, or not maintained, making it hard to evaluate the quality,safety, and usability of available routes. This lack of reliable information affects both occasional and regular cyclists, who may encounter poorly maintained paths, obstacles, or unsuitable routes without prior notice.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved